TUNISIAN REUBLIC
Ministry of Higher Education and Scientific Research
University of Carthage
National Institute of Applied Sciences and Technology

Graduation Project

to obtain the

National Engineering Diploma
in Applied Sciences and Technology

Specialty: Networking and Telecommunications

Development of an automated Internet monitoring and alert module for Dnext platform

---

Prepared by

Mohamed Ali Fathallah

Hosted by Dnext Intelligence

Defended on the 6th of July 2021 in front of a jury composed of:

Mr. Majdi JRIBI	President of the jury
Mrs. Dorsaf SEBAI	Reviewer
Mr. Maxime RIGAUD	Supervisor at the Company
Mrs. Wided MILED	Supervisor at INSAT

School year: 2020 / 2021

TUNISIAN REUBLIC
Ministry of Higher Education and Scientific Research
University of Carthage
National Institute of Applied Sciences and Technology

Graduation Project

to obtain the

National Engineering Diploma
in Applied Sciences and Technology

Specialty: Networking and Telecommunications

Development of an automated Internet monitoring and alert module for Dnext platform

Prepared by

Mohamed Ali Fathallah

Hosted by Dnext Intelligence

DNEXT

Supervisor at the Company Mr. Maxime RIGAUD	Supervisor at INSAT Mrs. Wided MILED
Date: June 18th, 2021	Date:

Visto Consulting SA
Av. Louis-Casaï 81
Case Postale 4
CH-1216 Cointrin

School year: 2020 / 2021

Dedication

I dedicate this work and give all my thanks to my family who never stopped supporting me through all my life, and especially my 18 years of studies.

My father Abdelhamid, my mother Aziza, my 4 awesome sisters Ahlem, Marwa, Khaoula and Safa thank you for being there for me. I will make you proud.

A huuuuuuuuge thanks to all my friends who made me feel so special on different occasions, Thank you for every moment, for every smile, you've been there for me when I needed you the most, I love you all.

PS: I'm starting a streaming career so follow me XD : twitch.tv/mafstar1

Acknowledgments

First of all, I would like to thank the Dnext family for giving me this opportunity to conduct my graduation internship within their organisation. It was a really good experience from which I've learned a lot. I would like to give special thanks to Mr. Sami JABALLAH and Mr. Guillaume GOUDY for trusing me and welcoming me in their company. Special appreciation to Mr. Maxime RIGAUD without whom this project couldn't be possible. He was an excellent supervisor and a very nice person.

Special thanks to Mrs. Wided MILED, my academic supervisor for supporting and helping me during this project and for being a perfect professor on both teaching and kindness levels.

I would like to thank all my professors in my entire 18 years of studying, without whom I wouldn't be able to reach this step. Special thanks to the INSAT family as a whole for providing the best environment for students to thrive and reach high levels.

Finally, I would like to thank all the members of the jury for accepting to evaluate my work and provide their priceless feedback.

Abstract

This work concerns the improvement of the Dnext Intelligence platform by developing a new module, namely *Watcher*. This new module aims to allow the users of the platform to monitor websites, mainly those providing information and data on commodity markets news. Through this module, users are also notified when new records are posted or updated and can keep a history of those changes.

The project is developed throughout a series of stages starting by deducing the necessary requirements of the application and designing the envisioned solution. This study will be followed by an establishment of a detailed technical structure of the solution and finally the implementation and deployment of the application.

Keywords: AWS, Serverless, Node.js, React, Full stack, Terraform, API, MySQL, Dynamo.

Résumé

Ce travail concerne l'amélioration de la plateforme Dnext Intelligence par le développement d'un nouveau module *Watcher*. Ce nouveau module vise à permettre aux utilisateurs de la plateforme de surveiller les sites web, principalement ceux qui fournissent des informations et des données sur l'actualité des marchés des matières premières. Grâce à ce module, les utilisateurs sont également avertis lorsque de nouveaux enregistrements sont publiés ou mis à jour et peuvent conserver un historique de ces changements.

Le projet est développé à travers une série d'étapes commençant par l'analyse des spécifications nécessaires de l'application et la conception de la solution envisagée. Cette étude sera suivie par l'établissement d'une structure technique détaillée de la solution et enfin la mise en œuvre et le déploiement de l'application.

Keywords: AWS, Serverless, Node.js, React, Full stack, Terraform, API, MySQL, Dynamo.

Contents

Abstract **i**

Résumé **ii**

list of figures **vi**

list of tables **viii**

List of acronyms **ix**

Introduction **2**

1 Project context **3**

1 Host company presentation 4

1.1 Company description 4

1.2 Platform description 5

2 Project scope 6

2.1 General context 7

2.2 Problem statement 7

2.3 Project goal 7

2.4 Terminology 7

3 Methodology 8

3.1 Agile methodologies 8

3.2 The SCRUM methodology 9

4 Modeling language 9

2 Software requirements specifications **11**

1 Current version of the platform . . . 12
2 Envisioned solution . . . 13
3 Actors identification . . . 14
4 Requirements specifications . . . 14
4.1 Functional requirements . . . 14
4.2 Non-Functional requirements . . . 15
5 Detailed requirements specifications . . . 16
5.1 Global system features . . . 16
5.2 Detailed description of use cases . . . 18
5.2.1 Create a new Watcher . . . 18
5.2.2 Create a new Focus Point . . . 20
5.2.3 Manage Watcher . . . 22
5.2.4 Manage Focus Point . . . 24

3 Design and architecture 26
1 Project general architecture . . . 27
1.1 Microservice architecture . . . 27
1.2 Project architecture . . . 29
1.2.1 Presentation tier . . . 29
1.2.2 Logic tier . . . 31
1.2.3 Data tier . . . 31
2 Detailed design . . . 32
2.1 Package diagram . . . 32
2.2 Class diagram . . . 33
2.3 Database schema . . . 35
2.4 Sequence Diagram . . . 35
2.4.1 Login Sequence Diagram . . . 35
2.4.2 Watcher creation/execution sequence diagram . . . 36

4 Realization and testing 39
1 Software environment . . . 40
1.1 Integrated Development Environment (IDE) . . . 40

1.2 Database design tool . . . 40
1.3 Version Control System (VCS) . . . 40
2 Development technologies . . . 41
2.1 Presentation tier technologies . . . 41
2.1.1 React.JS . . . 41
2.1.2 Ant Design . . . 42
2.1.3 HTML5, React Bootstrap and SCSS . . . 42
2.2 Logic tier technologies . . . 43
2.3 Data tier technologies . . . 44
3 Tests . . . 44
4 Application presentation . . . 45

5 Software deployment **50**
1 Deployment environment . . . 51
1.1 AWS Serverless framework . . . 51
1.2 Project architecture . . . 52
1.2.1 Presentation tier . . . 53
1.2.2 Logic tier . . . 54
1.2.3 Data tier . . . 58
2 Deployment tool . . . 58

Conclusion and perspectives **60**

A Appendix 1: Use cases description **61**

A Appendix 2: Platform presentation **64**

List of Figures

1.1 Host company logo 4
1.2 The main Process of the Platform 5
1.3 The Home page of the Dnext Platform 5
1.4 The Ingest Workflow page of the Dnext Platform 6
1.5 The Dashboards page of the Dnext Platform 6

2.1 Watcher's components. 15
2.2 Global Use Case 16
2.3 Create Watcher Use Case 18
2.4 Create Focus Point Use Case 20
2.5 Manage Watcher Use Case 22
2.6 Manage Focus Point Use Case 24

3.1 General Microservice Architecture 28
3.2 Project General Architecture 29
3.3 Dnext APP Client Architecture 29
3.4 Monitoring Function Package Diagram 32
3.5 Monitoring Function Class Diagram 34
3.6 Entity Relation Diagram 35
3.7 Login sequence diagram 37
3.8 Watcher Creation/Execution sequence diagram 38

4.1 DOM hierarchy in an HTML document. 42
4.2 Links List Focus Point Tests. 44
4.3 Main page of the Monitoring Module. 45
4.4 Watchers List Page. 45

4.5 Watcher Creation Form 46
4.6 Adding Schedules Page. 46
4.7 Adding Notifications Page. 47
4.8 Focus Point Creation Form. 47
4.9 Email Notification. 48
4.10 Focus Point Execution History. 49
4.11 Focus Point Values History. 49

5.1 General AWS Serverless architecture 52
5.2 Project General Architecture 52
5.3 Dnext APP Client Architecture 53
5.4 Dnext API Architecture 56
5.5 SQS queue integration 57

A.1 Watcher Details Page. 65
A.2 Element To Monitor. 65
A.3 Focus Point Result Preview. 66
A.4 Focus Point Details. 66

List of Tables

1.1 Comparison of agile methodologies 8
1.2 SCRUM Actors 9

2.1 Global use case diagram description 17
2.2 Create Watcher use case textual description 19
2.3 Create Focus Point use case textual description 21
2.4 Manage Watcher use case textual description 23
2.5 Manage Focus Point use case textual description 25

5.1 The difference between REST and Websocket APIs 55

A.1 Create Watcher use case diagram description 62
A.2 Create Focus Point use case diagram description 63

List of acronyms

- **API** Application Programming Interface
- **AWS** Amazon Web Services
- **BI** Business Intelligence
- **CDN** Content Delivery Network
- **DDoS** Distributed Denial-of-Service
- **DOM** Document Object Model
- **DNS** Domain Name System
- **EC2** Elastic Compute Cloud
- **HCL** HashiCorpConfiguration Language
- **HTML** HyperText Markup Language
- **HTTPS** Hypertext Transfer Protocol Secure
- **IT** Information Technology
- **IaC** Infrastructure as Code
- **NPM** Node Package Manager
- **RAD** Rapid Application Development
- **RDBMS** Relational Database Management System
- **REST** Representational state transfer
- **RUP** Rational Unified Process
- **S3** Simple Storage Service
- **SaaS** Softwate as a Service
- **SCSS** Sassy Cascading Style Sheets

- **SES** Simple Email Service
- **SQL** Structured Query Language
- **SQS** Simple Queue Service
- **UML** Unified Modeling Language
- **VCS** Version Control System

General Introduction

We are living in the age of technological progression. Digital advancements have completely revolutionized our everyday lives, and one of the largest impacts felt has been in the business world. Companies now have access to data-driven tools and strategies that allow them to learn more about their customers and themselves than ever before by profiting from the Business Intelligence (BI) technologies.

Business intelligence combines business analytics, data mining, data visualization, data tools and infrastructure, and best practices to help organizations to make more data-driven decisions. In practice, you know you've got modern business intelligence when you have a comprehensive view of your organization's data and use that data to drive change, eliminate inefficiencies, and quickly adapt to market or supply changes.

For instance, the agricultural commodities market has a huge need for technology and Big Data. Every commodity player is recollecting the same information, recreating the same database from many fragmented data in the format of reports and PDFs that are neither structured nor searchable. In that regard, comes Dnext Intelligence as a solution to empower the commodity players with market intelligence by providing data, tools of data management and analysis, forecast management and collaboration in a secured environment.

But one need remains unanswered, commodity players need to stay up to date and aware of every news and records as soon as they are published. This was done manually which was arduous and inefficient. Thus, our graduation falls within this context, aiming to develop an automated Internet monitoring and alert module for the Dnext platform. This module, called *Watcher* allows Dnext's users to monitor websites using different filters options, keep records of changes and get notifications when they occur.

This report summarizes and presents the different stages of the project's life cycle. It is organised ad follows:
In the first chapter, *Project context*, we will present the host company and give an overview of the project. The next chapter *Software requirements specifications* will start with the presentation of the current version of the platform and the envisioned version, followed by identifying the actors and establishing the requirements specifications. The third chapter *Design and Architecture* will describe the project general architecture and its detailed design. Then we move to the fourth chapter *Realisation and testing* where we will present the software environment and the development technologies used, then show the tests to finally present the application. The last chapter will be the *software deployment* which presents the deployment environment and tool used to deploy the project.
We conclude this report by a general conclusion that summarizes our work and presents some future perspectives.

1

Project context

Introduction

This first chapter aims to define the general context of the project, by introducing the host organization, its functional structure and its activity sectors. This chapter will also explore the needs driving the development of this project as well as the methodology adopted during its life cycle.

1 Host company presentation

In this Section, we will present the host company and then give a description of its platform.

1.1 Company description

This project was carried out by Dnext Intelligence [1] , a Swiss company, founded in 2020 and located in Geneva. Dnext Intelligence provides a highly evolutive commodity market intelligence platform, providing all needed tools and data to enhance clients market intelligence and risk management. Its mission is to empower the commodity players with market intelligence by providing data, tools and collaboration in a secured environment. Data is collected from dispersed public and private databases, such as government information or customs. Furthermore, Dnext's mission goes far beyond simply providing data. Indeed, Dnext takes advantage of the new possibilities offered by data science to develop unique analytical features, hence pushing the forecasting and data sharing options within companies and between them.

Figure 1.1: Host company logo

Our hosting company offers its services to clients in the following areas of expertise :

- Commodity trading companies
- Hedge funds, asset managers
- Producers and Food manufacturers
- Governments and public institutions
- Service industry (eg. inspection, freight, logistics ans insurance)

1.2 Platform description

The main process of the Dnext platform is illustrated in Figure 1.2. The first step of this process is data collection and aggregation. This collected data will be used later on with the analytic tools, taking advantage of the Artificial Intelligence (AI) and Machine Learning (ML) algorithms to improve forecasts. More details about the functionalities of this plateform will be given in the next chapter, section 1.

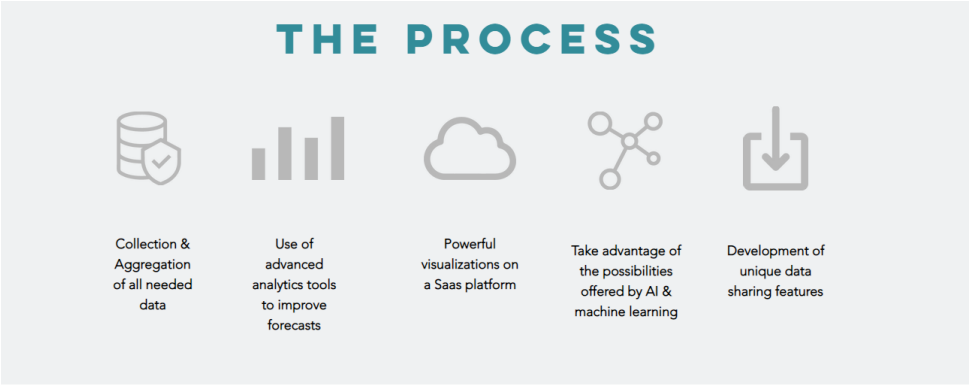


Figure 1.2: The main Process of the Platform

Furthermore, the Dnext platform offers many functionalities, powerful visualisation tools and data sharing features. Figure 1.3 shows the home page of the Dnext platform, and Figure 1.4 shows the ingest workflow module, which allows the user to ingest raw data and transform it into a structured datasets. As for the Figure 1.5, it shows the dashboarding module of the Dnext platform, which gives the user diverse visualization tools.

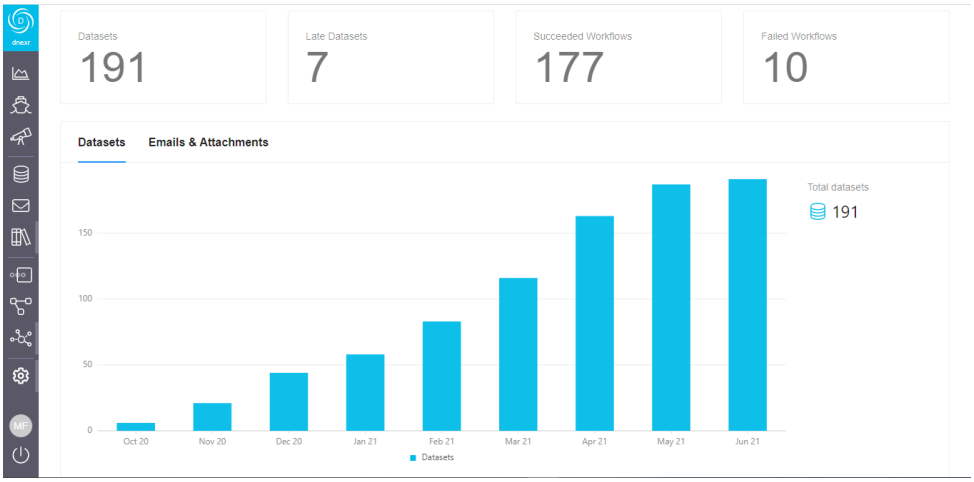


Figure 1.3: The Home page of the Dnext Platform

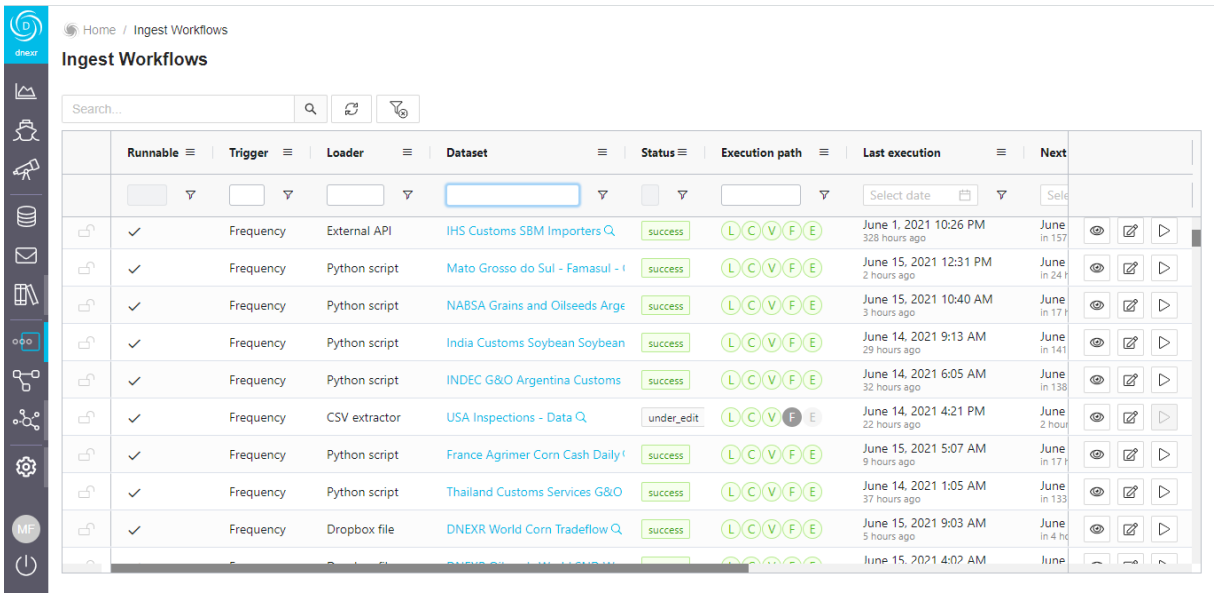


Figure 1.4: The Ingest Workflow page of the Dnext Platform

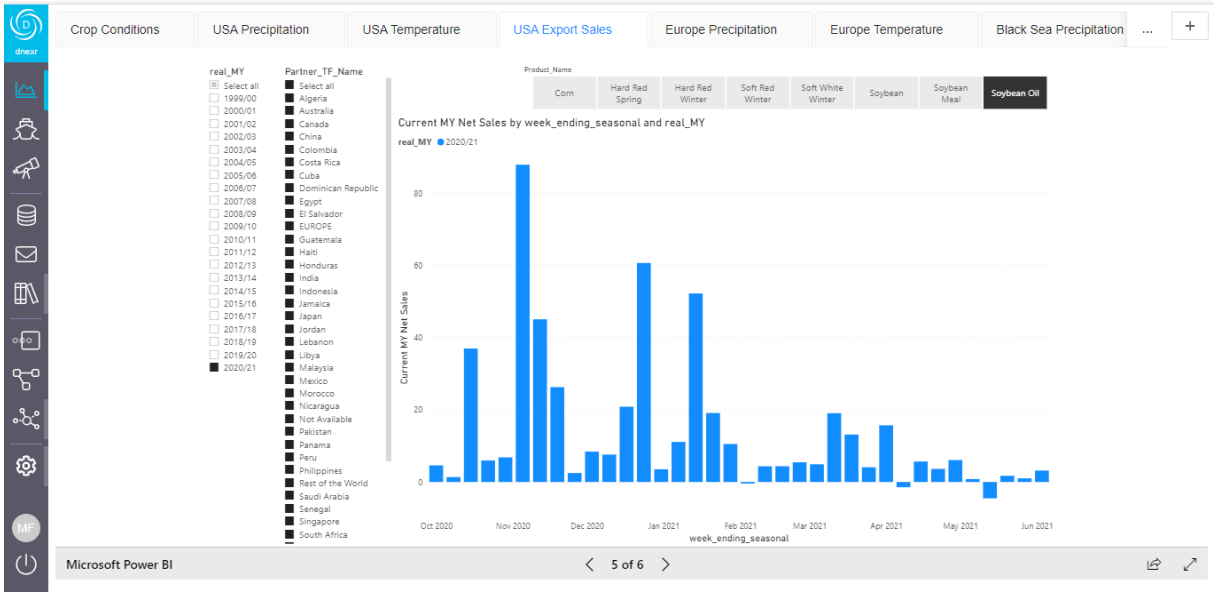


Figure 1.5: The Dashboards page of the Dnext Platform

2 Project scope

This section is a general presentation of the project scope. It will start by its context and key motives, followed by problem statement and ending with the project goal.

2.1 General context

The commodities market is a vast area of competition where there is massive amount of data to be considered and relied on to make decisions. Dnext intelligence comes as a global and complete solution for analytics by providing a unique solution with data Pull and Push capabilities, as well as better data management and visualization.
Being up to date with all the news regarding the market and having access to all records, as soon as they are published, are very crucial to have a full scope for the trading flows.

2.2 Problem statement

In order to profit from all the services provided by Dnext, analysts have to collect a massive amount of data and ingest it into the platform. Fortunately, there are dozens of data and news providers that publish their records online. However, it is still up to the user to monitor manually all of their websites to get them, and that is not efficient in a market where the speed of getting the information makes a huge difference. Regarding this issue, the development of a module that will monitor automatically certain websites chosen by the user and that detects the needed changes, alert about them and saves their history, became more and more essential. Our project is within this framework and consists of developing this automated monitoring module.

2.3 Project goal

With the mentioned requirements and issues considered, the objective of this internship is the development of a module that will allow the users of Dnext platform to monitor websites by creating *focus points* that define the exact elements to be monitored and provides an alerting system of all the changes that occur.

2.4 Terminology

- **Focus Point:** a specific element in a web page to be monitored using a combination of a set of filters.
- **Watcher:** regroups multiple focus points under the same category and defines the schedules on which they will be executed and the notifications to be sent. We want to

note that *Watcher* designates also the name of the developing module.

- **Data Steward:** every Watcher has a data Steward which is a user who receives every information and notification about that watcher.
- **Analyst:** throughout this report, Dnext Users can be referred as Analysts.

3 Methodology

Software development methodologies are evolving continuously. The goal is to adapt to both the internal and external constraints of the environment that are necessary to the project. To meet the specifications of the project, this section presents the methodology adopted for the development of the project.

3.1 Agile methodologies

Nowadays, Information Technology (IT) projects are becoming more and more flexible. That is why there was a serious need for new management methodologies. Agile methods are essentially a group of practices for the management of IT projects [2] [3]. Those methods have an objective of getting the customer more involved in the project through its development stage in order to have continuous feedback on the application. Table 1.2 demonstrates a comparison between the main three agile methodologies, namely Scrum, RUP and RAD.

Table 1.1: Comparison of agile methodologies

	SCRUM	**RUP**	**RAD**
Life cycle	Iterations called sprints. A sprint lasts up to a month and contains all phases.	The phases for one iteration: inspection, elaboration, construction and transition.	The simple development cycle: framing, design, construction and finalization. The duration is between 90 and 120 days.
Characteristics	Fairly simple approach.	Rather cumbersome approach.	Rapid approach.
Suitable project	Large or medium size projects.	Large or medium size projects.	Projects with urgent requirements.

In this project, the choice has been made for the agile methods since they offer greater risk management, visibility and adaptability. Otherwise, by having a small project team working on several projects in parallel, the proper approach needs to be both flexible and simple which led to the choosing of the SCRUM methodology, this methodology will be explained in the next section.

3.2 The SCRUM methodology

With Scrum methodology, projects are divided into sprints with the goal of producing a functional software by the end of each sprint. The client works side by side with the project team to schedule the tasks according to their priorities. At the end of each sprint, within a meeting known as the sprint retrospective, the project team evaluates the previously finished sprint. This meeting is essential to assess the productivity of the team and the shortcomings of the previous sprint. The principal advantage of SCRUM is its flexibility, since it allows the project team to maximize cooperation with the clients during the iterations [4].

Table 1.2: SCRUM Actors

Actor	**Description**
Product Owner (Bastien Lambert)	their main roles is to manage the Product Backlog and organise the team efforts in order to improve the product value
Development Team (Mohamed Ali Fathallah)	a group of professionals who are capable of delivering a functional deliverable by the end of each sprint.
SCRUM Master (Maxime Rigaud)	the person who ensures the smooth running of the sprints and the project in general.

4 Modeling language

In order to design and model this project, the choice was made on using UML: Unified Modeling Language. This modeling language allows the description of both needs and details of the architecture. The UML diagrams represent the application within its system and how it is integrated. In the next chapters, there will be a use of this modeling language to show the design of the solution as well as to define its interaction with its software environment.

Conclusion

This chapter presented the project's general scope. It also defined the chosen methodology which was used during the development of the solution. The next chapter will present the requirements of the project as well as the architecture of the solution.

2

Software requirements specifications

Introduction

The second chapter gives a complete description of the solution's behavior. First, we present the current version of the platform and its problems. Secondly we explain the envisioned solution and the features that should be provided. Then, we identify the different actors that will interact with the application and the general overview of functional and non-functional requirements. Finally, we will go in depth and fully detail our functional software requirements.

1 Current version of the platform

At the time of the launch of this project, the existing Dnext Itelligence platform offers many services. We give, below, a brief overview of each service.

- **Data Management:** It is one of the major building blocs of the platform. It helps to manage three types of data categories:
  - SND - Supply and Demand: which is composed of crop production (Weather and satellite, economics. Margins, etc), domestic demand, TradeFlow (Import, Vessel tracking, Seasonality)
  - Prices
  - Non-Fundamental analysis (volumes, technical...)

  The platform provides different processes to perform:
  - Data loads (Excel, pdf, Web page, APIs, etc.)
  - Data Validation
  - Data transformation
  - Data aggregation
- **Data Analysis:** It is the second major building bloc of the platform. Once data has been cleansed and normalized, the users will have the ability to analyse the stored data sets. Thus, the following possibilities are provided:
  - Reporting and dashboarding at different time scales (daily, weekly, monthly, quarterly and yearly)
  - Ability to link platform data to Excel-like solutions

  Before data analysis, different processes are needed :
  - *Data conversion* : by commodity and origin, by commodity and destination - any unit conversion depending on commodity and country
  - *Temporal aggregation* : from weekly data to monthly, monthly to quarterly, of first half and second half, etc.
  - *Priority rules filtering* based on Origin, Commodity and Time period.

  - *Quantity aggregation* : by commodity / geography or group of commodities / geographies.
  - *Data source ordering* : Actual then Pending, Corrections then forecast, etc.

- **Forecast Management**
  - Ability to create forecast data for a specific data set.
  - The forecast could be entered: monthly, quarterly or yearly.
  - Then, the system will split the values to the minimum frequency.
  - The forecast could be entered manually or based on last year seasonality or two years seasonality or other rule.

The main problem of the Dnext platform process is that most of the data collection has to be done manually by the user of the platform. Thus, this latter has to keep monitoring by himself all the data sources, mainly the websites of the records providers.
Hence, developing a module to automate this workflow becomes a necessity. This module will monitor certain websites that publish news or trade flow records and will alert platform's users whenever there are some changes with the ability to keep track of all the history of them.

2 Envisioned solution

During the span of the internship, the main focus was to overcome problems mentioned in the previous section, and that is by:

- Developing a new module, which will be called by the Dnext team *Watcher*, allowing the users of Dnext platform to monitor websites by creating *focus points* (defined in the previous chapter), providing an alert system of all the changes in these focus points and saving them to have a full history. This module will keep the analysts up to date with all the news and records and everything important in the commodities market.
- Writing scripts to scrape data from different sources (websites, pdfs, xls files, etc.) provided by the records publishers or the clients themselves, to be able to use it to load the platform data sets.

3 Actors identification

Actors are defined as entities that play a role in the system. An actor can be a user, a machine or a process that interacts with the main system. In this project, there is only one actor:
Dnext User: the user of the Dnext platform who is a client that has full access to the platform and all its features. He is usually a data analyst in the commodities market.

4 Requirements specifications

To successfully complete the project, it is essential to elaborate the functional and non-functional requirements, which will ensure consistency between the client's requirements, and the architecture to be designed.

4.1 Functional requirements

In order to satisfy all the previous needs and solve the clients issues, the application must include these functionalities:

- **Creating Watchers:** Allow the user to create a *Watcher* which will monitor and supervise websites according to defined schedules and using specific *Focus Points* to detect news or changes and then send notifications to certain users if that occurs.
- **Creating Focus Points:** Allow the user to create *Focus Points*, which are the specific elements to monitor on a web page. They are defined by a monitoring types and a combination of filters and can be titles, links, texts or numeric values, etc. .
- **Managing Watchers:** Allow the user to modify any of the already created watchers attributes, including schedules, notifications, focus points, or delete the watcher.
- **Managing Focus Points:** Allow the user to modify any of the already created focus points attributes, including the monitoring type, the URL, the filters... or delete the focus point.
- **Watchers dashboard:** Display a dashboard of the created watchers containing stats and updates.

- **Executions and changes history:** Allow the user to access to all the executions history of every watcher and all values recorded from each focus point.

Figure 2.1 illustrates the main components of a watcher.

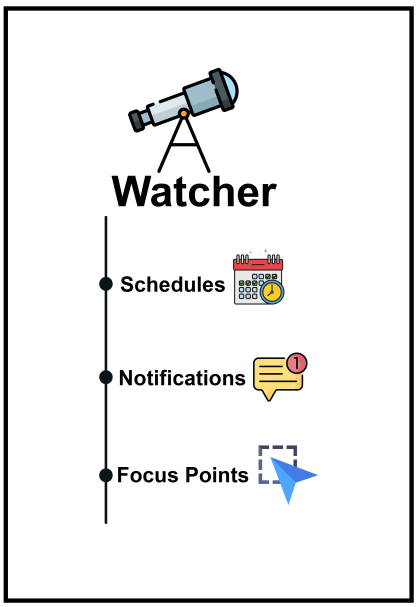


Figure 2.1: Watcher's components.

4.2 Non-Functional requirements

This project was carried out, primarily, to answer new needs and resolve concerns related to the user experience. Therefore, it is mandatory to detail all the non-functional requirements that will play roles as important as the functional ones, in this project:

- **Scalability**: Dnext Platform is constantly evolving. Features are added to guarantee maximum satisfaction to its users. So scalability must be ensured on two levels:
  - Application architecture: an extensible architecture should be used in the development of the application to minimize conflicts and effort required to add new features.
  - User interface: the interface must allow adding elements without cluttering up what exists and without changing the usual location of the elements, reducing the time for users to adapt to changes.
- **Ergonomics**: users are not necessarily technologically trained, therefore we need to create user friendly and responsive interfaces.

- **Documentation**: documentation is mandatory to better track everything and always have a full understanding of all the moving parts. It also greatly reduces the code base's learning curve.

5 Detailed requirements specifications

In this section, we give a global view on the functional behavior for the actor, by presenting the use case diagrams and then detail the most prominent use cases.

5.1 Global system features

Figure 2.2 illustrates the global use case diagram which gives an overall description of the main features of the application.

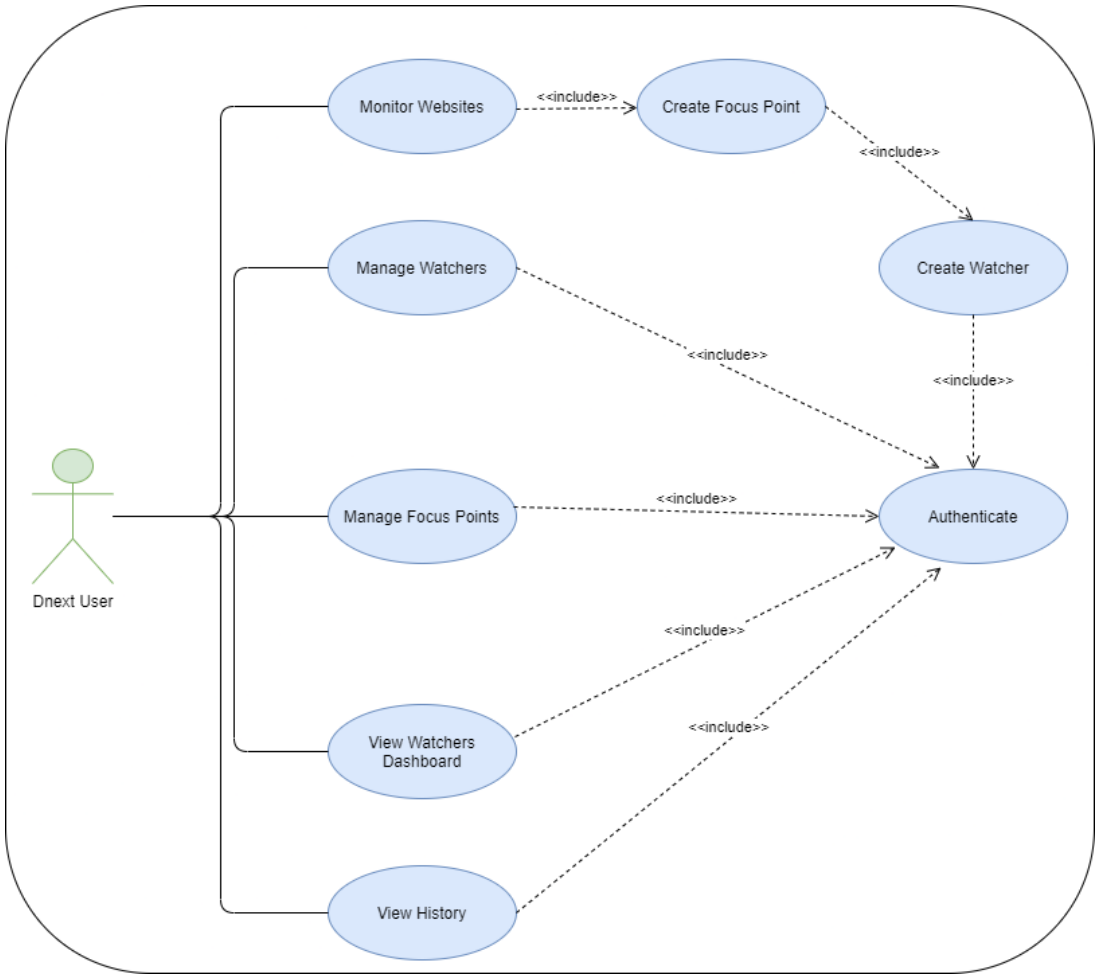


Figure 2.2: Global Use Case

These different use cases are general. Therefore, they can be decomposed into more unitary use cases. Table 2.1 gives a detailed description of each use case:

Table 2.1: Global use case diagram description

Use Case	**Description**	**Actor**
Monitor Websites	The user can monitor different websites by creating Watchers and adding Focus Points in them.	Dnext User
Create Watcher	The user can create a Watcher by defining its name, its Criticality level, its data steward, adding the desired notifications and fixing the schedules according to which the watcher will be executed.	Dnext User
Create Focus Point	The user can create a Focus Point by defining its name, monitoring type and the URL of the website to monitor and finally choosing the right combination of filters to select the exact element he needs.	Dnext User
Manage Watchers	The user can modify any of the already created watchers by changing its attributes(name, data steward, level of criticality), adding or removing schedules, notifications, focus points.or delete the watcher itself.	Dnext User
Manage Focus Point	The user can modify any of the already created focus points by changing its attributes (The monitoring type, the URL, the filters) or delete the focus point itself.	Dnext User
View Watcher Dashboard	The user has access to a dashboard containing stats and updates about the active watchers	Dnext User
View History	The user has access to all the executions history of every watcher with the execution result(success or error) and the changes history,every unique values recorded from each focus point.	Dnext User

5.2 Detailed description of use cases

This subsection will give further details and the nominal flow for the main use cases mentioned above.

5.2.1 Create a new Watcher

Figure 2.3 illustrates the Create Watcher use case diagram, giving more details about it.

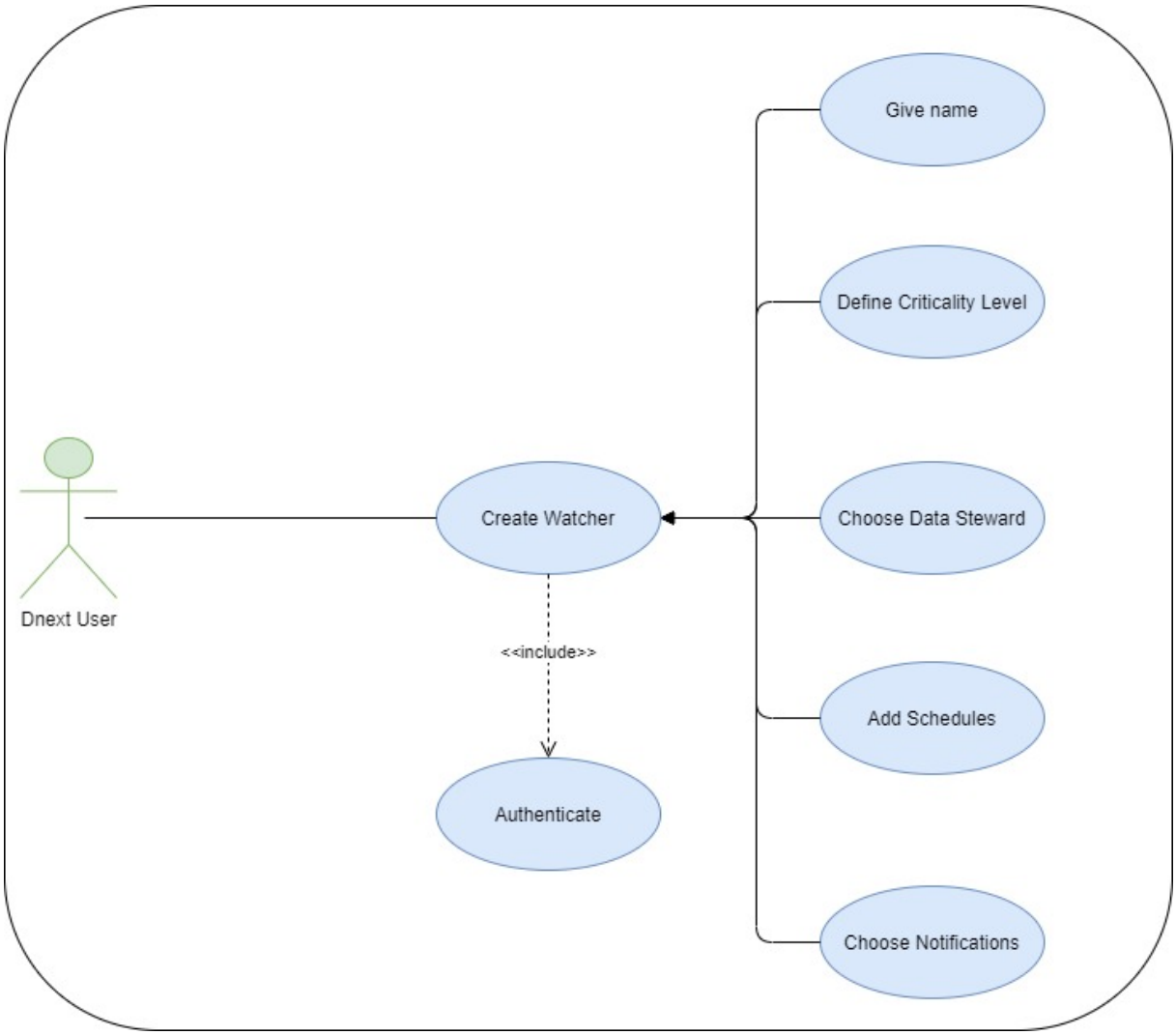


Figure 2.3: Create Watcher Use Case

Table A.1 in Appendix 1 gives a full description to the Create Watcher use case diagram.

Table 2.2: Create Watcher use case textual description

Title	**Create Watcher**
Summary	The user create a new watcher to monitor some Focus Points on specific schedules and receive chosen notifications.
Actor	Dnext user
Pre-condition	The user navigates to the watcher main page or watchers list page .
Nominal scenario	1. User clicks on "creates a new watcher" button. 2. User types the Watcher's name. 3. User chooses the level of criticality. 4. User chooses the data steward. 5. User adds one or multiple schedules. 6. User adds one or multiple notifications. 7. User clicks on "save" button. 8. User is redirected to the watcher details page.
Alternative scenario	A7: User cancels the creation by exiting the page.
Exceptional scenario	E7: A required field is missing
Post-condition	A new Watcher is created

Note that in the textual descriptions, the letters 'A' and 'E' are used to say respectively 'Alternative' and 'Exception'. The number indicates the step at which this alternative scenario or exception may occur.

5.2.2 Create a new Focus Point

Figure 2.4 illustrates the Create Focus Point use case diagram:

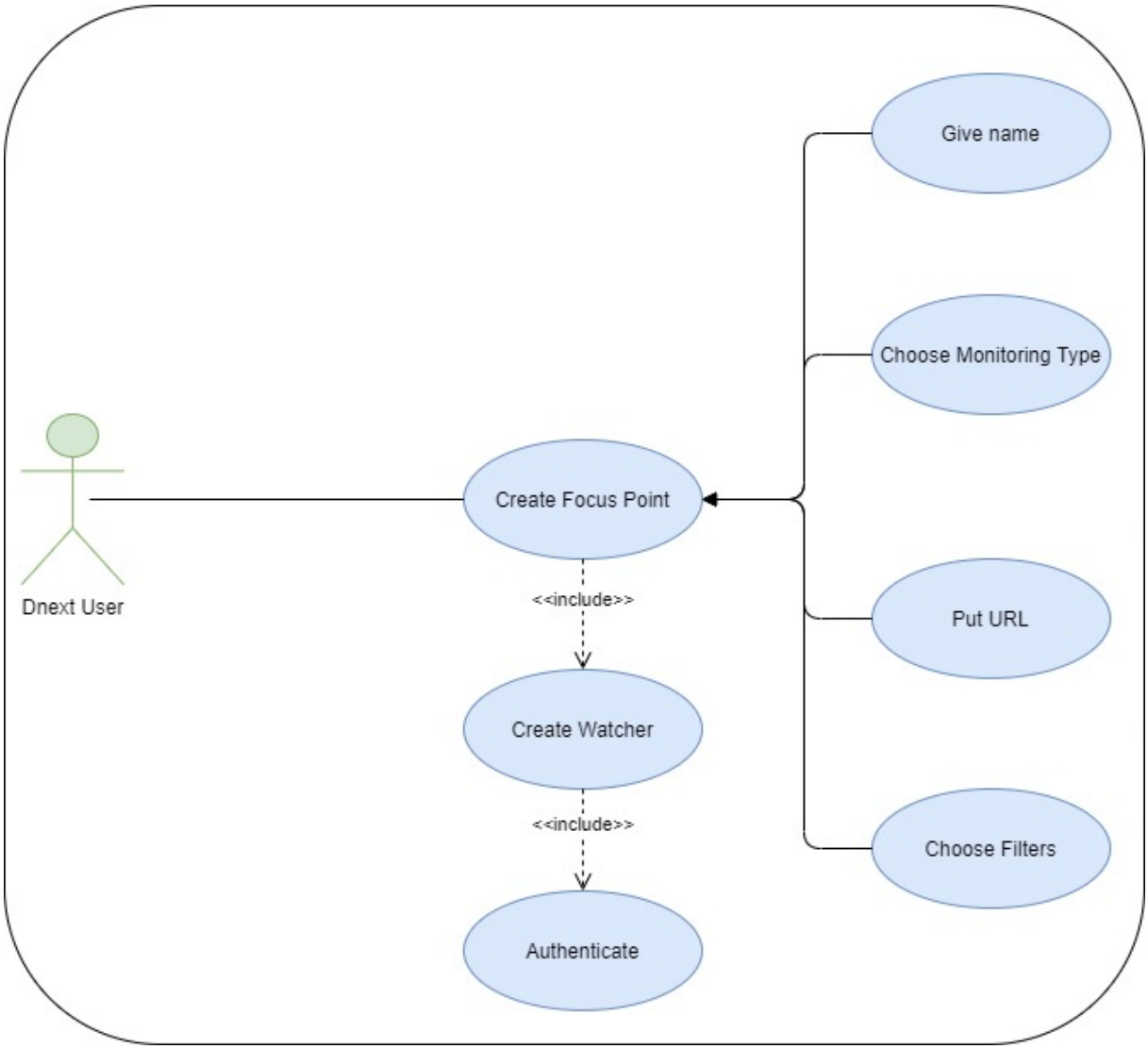


Figure 2.4: Create Focus Point Use Case

Table A.2 in Appendix 2 gives a full description to the Create Focus Point use case diagram.

Table 2.3: Create Focus Point use case textual description

Title	**Create Focus Point**
Summary	The user creates a new focus point associated to a watcher, it will define the specific element to monitor in a certain web page .
Actor	Dnext user
Pre-condition	The user navigates to the watcher details page .
Nominal scenario	1. User clicks on "create a new focus point" button. 2. User types the focus point's name. 3. User chooses the monitoring type. 4. User types the URL. 5. User chooses the necessary filters. 6. User clicks on "save" button. 7. User is redirected to the watcher details page.
Alternative scenario	A7: User cancels the creation by exiting the page.
Exceptional scenario	E7: The URL is not valid E7: A required field is missing
Post-condition	A new Focus Point is created

5.2.3 Manage Watcher

Figure 2.5 illustrates the Manage Watcher use case diagram, which gives it more details.



Figure 2.5: Manage Watcher Use Case

Table 2.4: Manage Watcher use case textual description

Title	**Manage Watcher**
Summary	The user can manage and change the attributes of the already created watcher.
Actor	Dnext user
Pre-condition	The user navigates to the watchers list page or the watcher details.
Nominal scenario	1. User clicks on "edit watcher" button. 2. User types the new Watcher's name. 3. User chooses the new level of criticality. 4. User chooses the new data steward. 5. User adds one or multiple schedules. 6. User adds one or multiple notifications. 7. User clicks on "save" button. 8. User is redirected to the watcher details page.
Alternative scenario	A7: User cancels by exiting the page. A7: User Deletes the watcher by clicking delete button.
Exceptional scenario	E7: A required field is missing
Post-condition	changes saved

5.2.4 Manage Focus Point

Figure 2.6 illustrates the Manage Focus Point use case diagram:

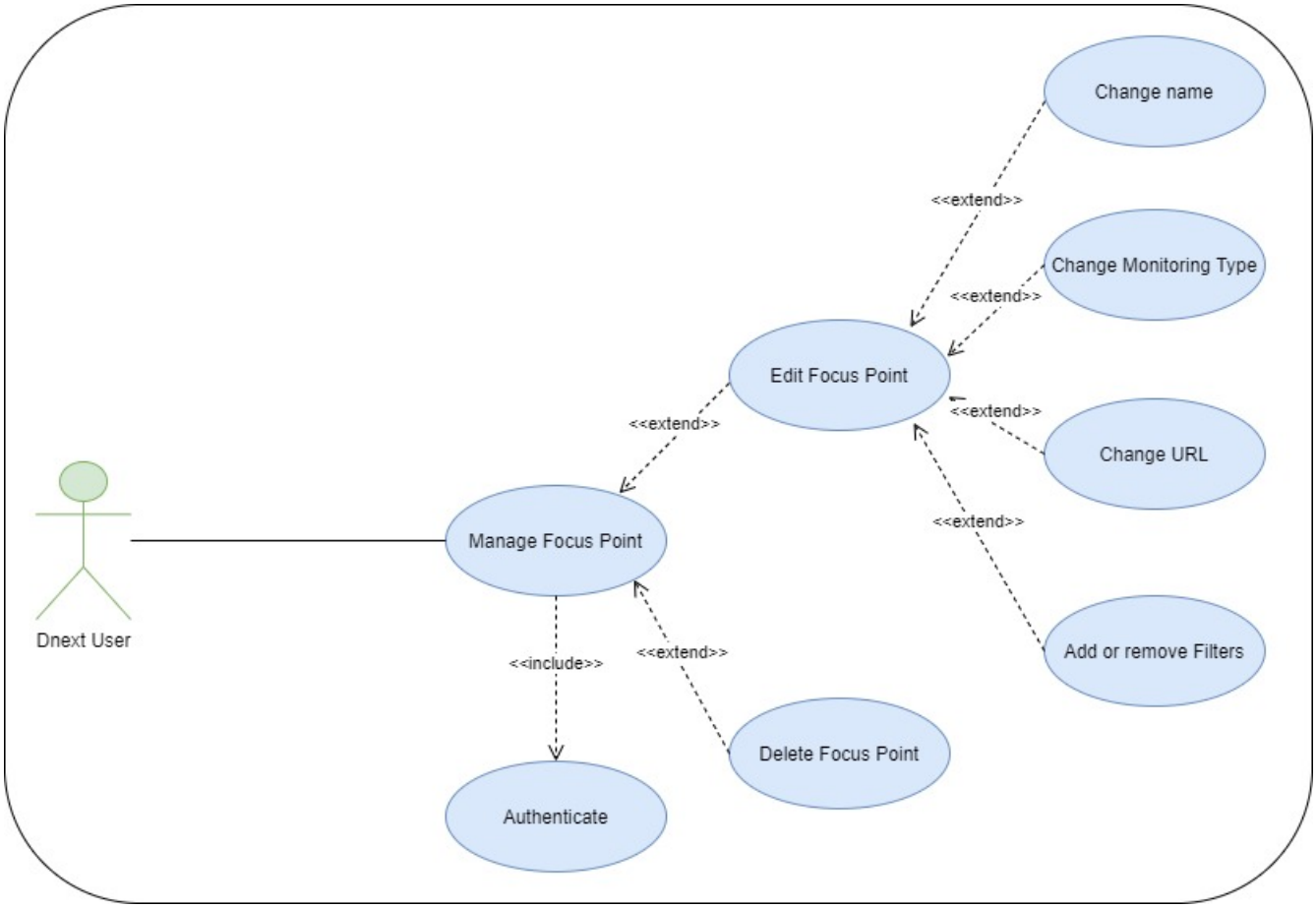


Figure 2.6: Manage Focus Point Use Case

Table 2.5: Manage Focus Point use case textual description

Title	**Manage Focus Point**
Summary	The user can manage an already created Focus Point by changing its attributes or deleting it.
Actor	Dnext user
Pre-condition	The user navigates to the Focus Point details page.
Nominal scenario	1. User clicks on edit button. 2. User types the focus point's new name. 3. User chooses the new monitoring type. 4. User types the new URL. 5. User chooses the new necessary filters. 6. User clicks on "save" button. 7. User is redirected to the watcher details page.
Alternative scenario	A7: User cancels by exiting the page. A7: User deletes the Focus Point by clicking delete button.
Exceptional scenario	E7: The URL is not valid E7: A required field is missing
Post-condition	changes saved

Conclusion

This chapter dealt with both functional and non-functional requirements of the application. It identified the main actors of the application and detailed some use cases. The next chapter will study the architectural side of the solution.

3

Design and architecture

Introduction

This chapter deals, fist, with the design of the solution by presenting the general architecture of the project, which is a three-tier architecture. Then, we will give details about this architecture using different diagrams, namely the package, class and sequence diagrams.

1 Project general architecture

The Dnext Platform is built with the Microservice architecture. This choice was based on many reasons:

- There is a team of developers working on the application.
- New team members must quickly become productive.
- The application must be easy to understand and modify.
- The need of Continuous deployment of the application.
- Scalability and availability requirements.
- Taking advantage of emerging technologies (frameworks, programming languages, etc)

In this section we will present the Microservices architecture and then elaborate on the architecture of our project.

1.1 Microservice architecture

Microservices also known as the microservice architecture is an architectural approach that builds an application as a series of interconnected services that are:

- Highly maintainable and testable: allows for frequent and quick development and deployment.
- Loosely coupled: permits a team to work on their service(s) alone for the most of the time without being disrupted by changes to other services or influencing other services.
- Independently deployable: allows a team to deploy their service without coordinating with other teams.
- Owned by a small team: By avoiding the high communication head of large teams, it is necessary for good productivity.

The microservice design enables huge, sophisticated applications to be delivered quickly, often, and reliably. It also enables a company technology stack to adapt [5].

Figure 3.1 illustrates the general architecture of Microservices.

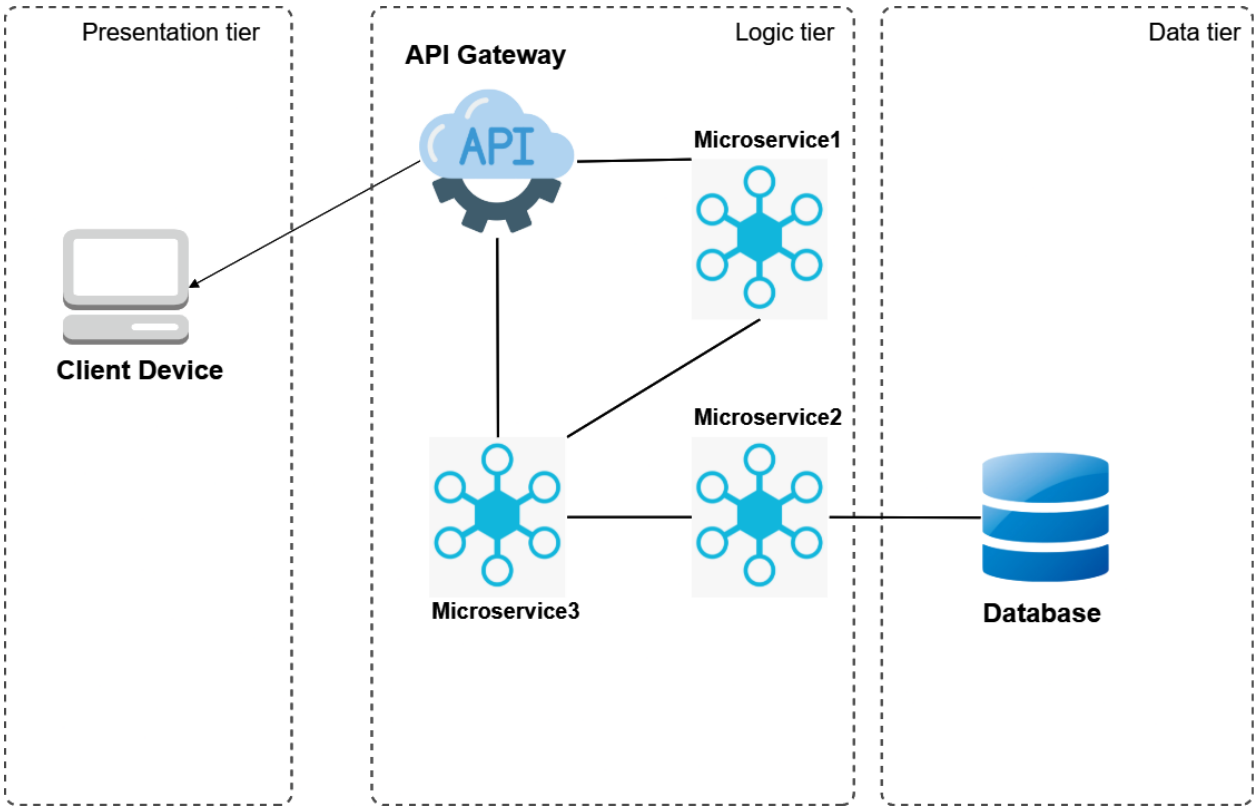


Figure 3.1: General Microservice Architecture

For decades, the multi-tier application (three-tier, n-tier, and so on.) has been a staple architecture pattern for user-facing applications, and it continues to be so. Although the terminology for describing a multi-tier architecture varies, a multi-tier application often includes the following elements [6]:

- **Presentation tier:** Component with which the user directly interacts (web page, mobile app UI, etc.)
- **Logic tier:** Code required to translate user actions to application functionality (CRUD database operations, data processing, etc.)
- **Data tier:** Storage media (databases, object stores, caches, file systems, etc.) that holds the data relevant to the application.

This Three-tier architecture can be applied to the Microservice architecture. We will further elaborate this approach in our application in the next subsection.

1.2 Project architecture

In this subsection, we will present the general architecture of the project and then explain each one of its tiers. Figure 3.2 illustrates the general architecture of the application.

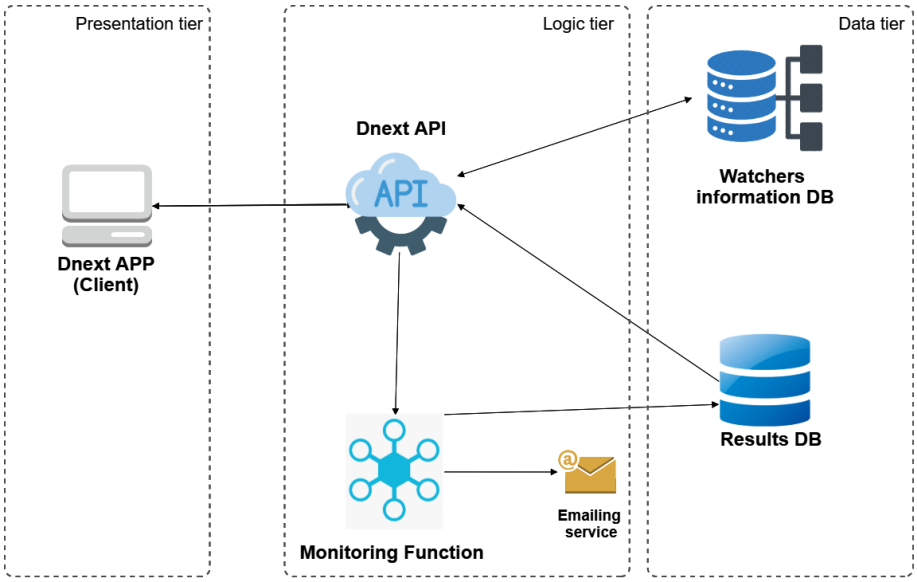


Figure 3.2: Project General Architecture

1.2.1 Presentation tier

The presentation tier consists of the Dnext platfrom web application, mainly the client side module. It contains the frontend of the project which is the user interface. Figure 3.3 illustrates the detailed architecture of Dnext APP client.

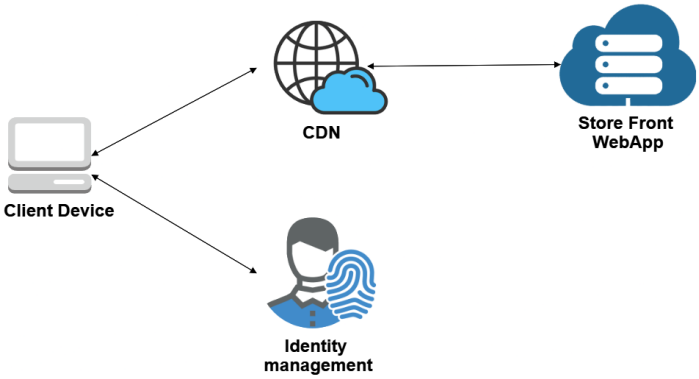


Figure 3.3: Dnext APP Client Architecture

The presentation in our project uses the following services:

Content Delivery Network: A content delivery network (CDN) is a geographically dispersed group of servers that collaborate to deliver Internet content quickly. It provides the rapid transfer of materials as HTML pages, javascript files, stylesheets, pictures, and videos that are required for loading Internet content. CDN services are becoming increasingly popular, and they now handle the bulk of web traffic, including traffic from large sites like Facebook, Netflix, and Amazon. A correctly built CDN can also assist to protect websites from malicious assaults like Distributed Denial of Service (DDOS) [7]. Using a content delivery network has many benefits:

- Improving website load times: The use of a nearby CDN server allows material to be distributed closer to website visitors, resulting in faster page loading time. A CDN can minimize bounce rates and enhance the amount of time people spend on a site because visitors are more likely to click away from a slow-loading site. In other words, a speedier website will attract more people who will remain longer.
- Reducing bandwidth costs: The cost of bandwidth use for website hosting is a major expense for businesses. CDNs can reduce the quantity of data an origin server needs to offer through caching and other improvements, lowering hosting costs for website owners.
- Increasing content availability and redundancy: Large quantities of traffic or hardware breakdowns might cause a website's usual operation to be disrupted. A CDN can manage more traffic and resist hardware failure better than multiple origin servers because of its dispersed nature.
- Improving website security: DDoS mitigation, improved security certifications, and other optimizations are some of the ways a CDN can increase security.

Store Front WebApp: It is a web hosting service used to host the platform's frontend static resources. Web hosting is allocating room on a web server for a website's files to be stored. It makes the files that make up a website (code, pictures, and so on) accessible online.

Identity Management: The organizational process of ensuring that individuals have proper access to digital resources is known as identity management (ID management). This comprises identifying, authenticating, and authorizing a person, or a group of people, to get access to applications, systems, or networks. This is accomplished by linking known identities with user privileges and limits. Software procedures that require access to

organizational systems are also referred to as managed identities. Identity management can be thought of as a critical component of security [8].

1.2.2 Logic tier

The application's brains are represented by the logic tier of the three-tier design. This is where Amazon API Gateway and the microservices can have the largest impact. They enable you to create a highly accessible, scalable, and secure application. Thus Our Project's Logic Tier consists of 2 components:

- **Dnext API:**
  API is the acronym for Application Programming Interface, which is set of programming code that enables data transmission between one software product and another. It defines also the terms of this data exchange.
  In the context of our project the Dnext API's main roles will be the management of data storing and loading, and the triggering of the Monitoring Function.
- **Monitoring Function:**
  It is a microservice that will run the code to execute the watcher. This Monitoring function will be triggered by the Dnext API and it will execute a specific watcher, by running every focus point in regard of its monitoring type and filters, then it will store the results in a database which is in the Data tier and send the necessary notifications using an emailing service.

1.2.3 Data tier

The Data Tier contains all databases where we store the significant data for the project. The data storage options available in the data tier are unaffected by the logic tier. By providing the proper database driver in the microservice, it can connect to any data storage choice.
Choosing a data store is highly dependent on the application requirements. In that regard, we chose to work with two types of data bases:

- **RDBMS:** To store the watchers, the focus points, the schedules, and all their attributes.
  RDBMS stands for Relational Database Management System. A relational database

organizes data into one or more data tables in which data types can be associated to one another, allowing the data to be structured.

- **NoSQL Data Base:** To store the executions history and their results (the values recorded for every focus point).
  A NoSQL database is not tabular and doesn't hold data in the same way that relational tables do. NoSQL databases are classified according to their data model. Document, key-value, wide-column, and graph are the most common types. They have adaptable schemas and can handle big amounts of data and high user loads with ease.

2 Detailed design

In this section, there will be a detailed illustration of the design of the application. First, there will be a diagram describing the different packages of the Monitoring function. Secondly, there will be the class diagram, Then the schema of the database will be presented. By the end of the chapter, there will be a sequence diagram to describe the behavior of certain entities in the project.

2.1 Package diagram

Figure 3.4 illustrates the Package diagram for the Monitoring function.

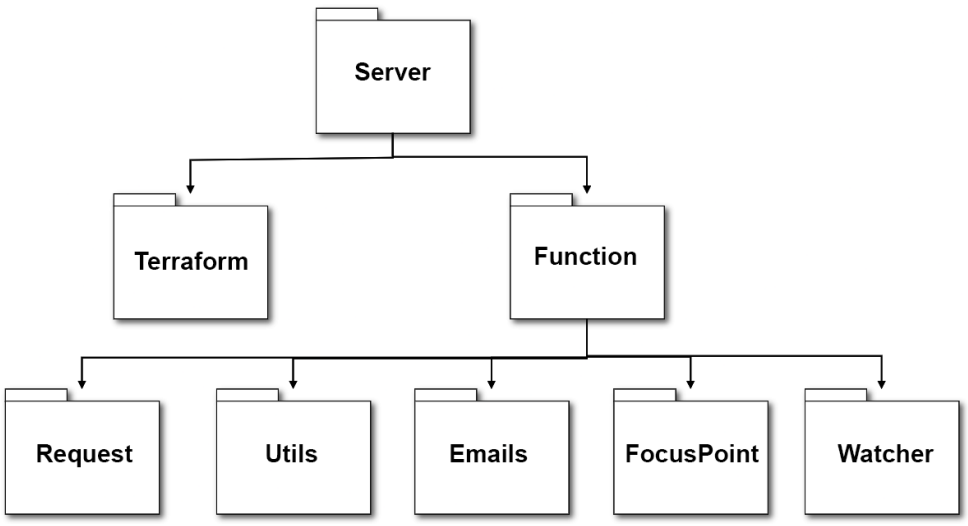


Figure 3.4: Monitoring Function Package Diagram

- **Server:** contains the main program that uses all the other packages.

- **Terraform:** contains the terraform configuration of the AWS and the database.
- **Function:** groups all the different packages to be used in the treatment to get the results.
- **Watcher:** Contains the watcher class.
- **FocusPoint:** contains the main FocusPoint Class and its children.
- **Email:** contains the creation of the email and its design.
- **Utils:** contains the various methods and functions shared between several classes in order to centralize certain common processes.
- **Request:** contains the main methods to send the get requests to fetch the HTML of the web pages.

2.2 Class diagram

Figure 3.5 illustrates the Class diagram for the Monitoring function.

- The Watcher class represents the watcher which has a list of focus points as an attribute.
- The FocusPoint class represent the focus point, its attributes are used to define the exact element to be monitored on a certain website and it has different methods to be used to get these parameters and get the results and compare them with old values.
- The BaseWebFocusPoint extends FocusPoint and it defines more methods to be used in the monitoring process by applying the filters and fetching the needed values.
- TextChanges class extends BaseWebFocusPoint, it represents the text changes monitoring type, it has one method getValue() which will do all the process needed to return the value of the monitored element.
- LinkChanges class extends BaseWebFocusPoint, it represents the Link changes monitoring type, it has one method getValue() which will do all the process needed to return the value of the monitored element.
- LinksListChanges class extends BaseWebFocusPoint, it represents the Links List changes monitoring type, it has one method getValue() which will do all the process needed to return the value of the monitored element.

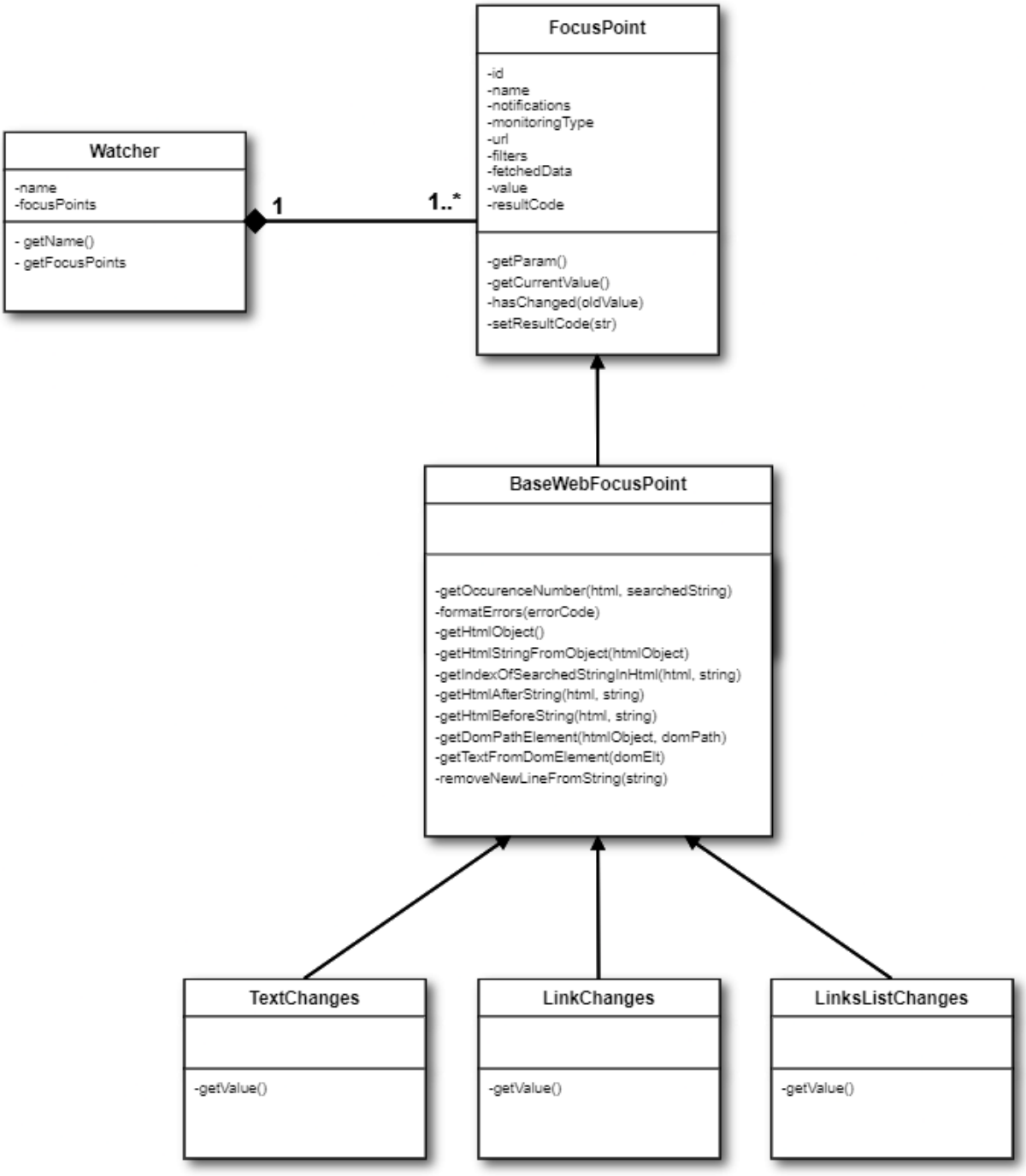


Figure 3.5: Monitoring Function Class Diagram

2.3 Database schema

Figure 3.6 illustrates the Entity Relation diagram for the project's MySQL database.

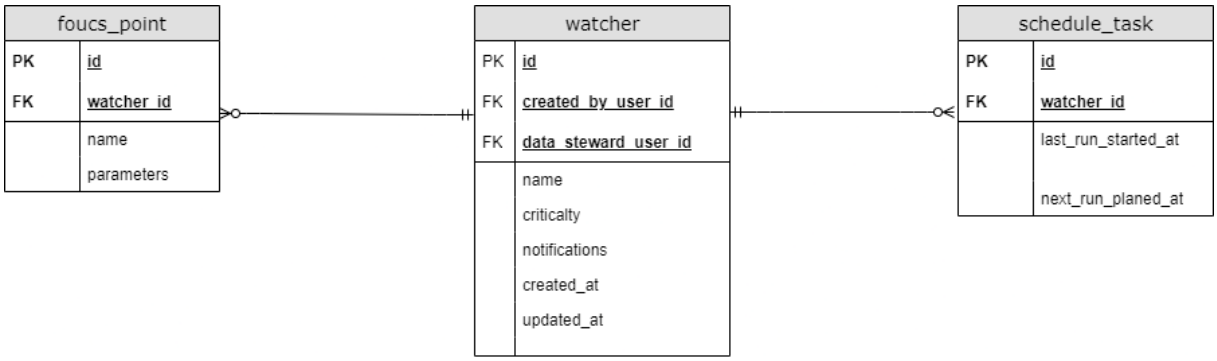


Figure 3.6: Entity Relation Diagram

As we can see this Entity Relation Diagram is pretty simple, as it only represents the tables related to our project not the platform as whole.
The relation between watcher and focus_point is One mandatory to many optional, as a watcher can have zero or many focus points, but a focus point must belong to one and only watcher.
The relation between watcher and schedule_task is One mandatory to many optional, as a watcher can have zero or many schedules, but a schedule must belong to one and only watcher.

2.4 Sequence Diagram

In this subsection, we will present the sequence diagram for the login and for the creation and execution of a watcher.

2.4.1 Login Sequence Diagram

To be able to use the Dnext services which include the monitoring module, the user must login and connect to the Dnext API. This process is managed by identity management system, as shown in the diagram. After the establishment of the connection, the front and the API can exchange Data to load the web pages.

Figure 3.7 illustrates the sequence diagram for the login.

2.4.2 Watcher creation/execution sequence diagram

To create a watcher, the user sends a PUT request to the API containing all its data. In return, the API stores them in the data base (same process applies for the focus point), and then when the scheduled time comes, the API triggers the microservice that manages the execution of that specific watcher. In our project, it's called Monitoring function. The trigger happens with a message which contains the parameters of the watcher and its focus points. After the execution, the monitoring function send notifications to the chosen users and then stores the results in an DB. The API fetches these results and sends them back to the front to be loaded on the web pages.

Figure 3.8 illustrates the sequence diagram for the creation and execution of a watcher.

Conclusion

This chapter described the architecture and design of the project. It presented at first the application architecture. Afterwards, it detailed the design through various UML diagrams. The next chapter will show the realization and tests of the application.

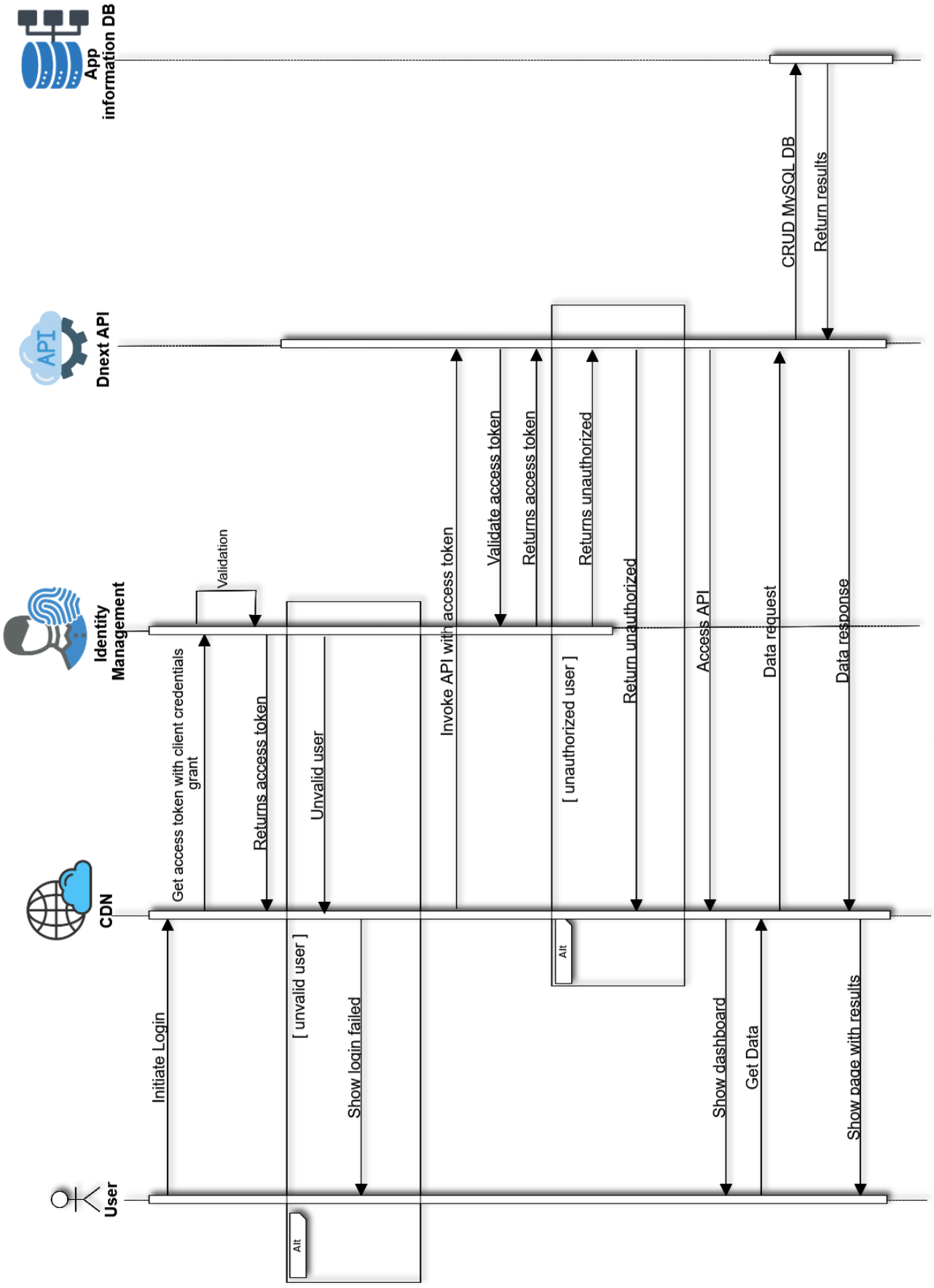


Figure 3.7: Login sequence diagram

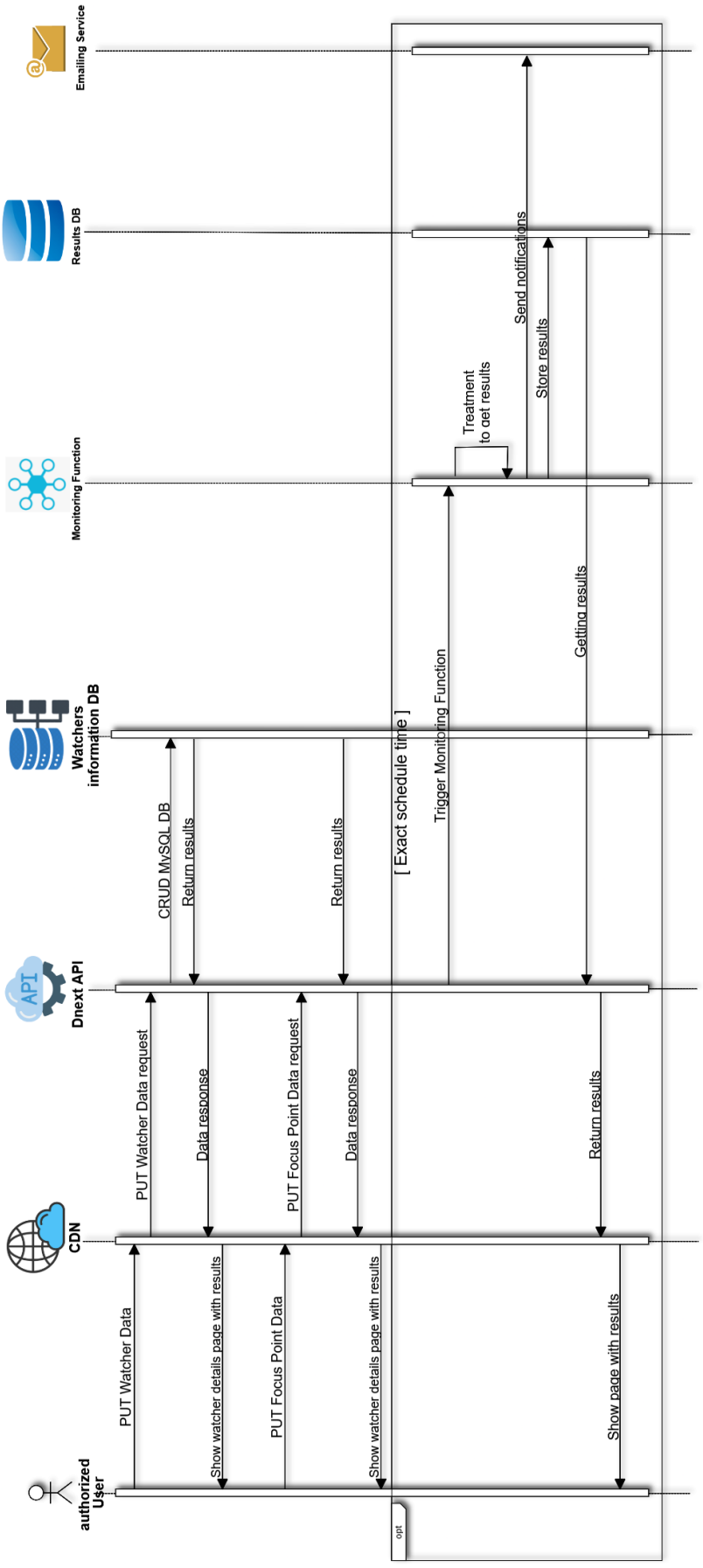


Figure 3.8: Watcher Creation/Execution sequence diagram

4

Realization and testing

Introduction

Now that the application design has been detailed, this chapter will deal with the realization of the solution. first, it will describe the software environment used. Next, it will move on to the technologies used for the development of the different entities of the project. Then, it covers the set of tests performed on the solution. And finally, there will be a presentation of key scenarios of the applications.

1 Software environment

To create a stable development environment, a reliable network of technologies is necessary. In this section, we will present the different software environments and tools used during the project.

1.1 Integrated Development Environment (IDE)

As an IDE, we used Microsoft's Visual Studio Code. It is a source-code editor for Windows, Linux, and macOS. Debugging, syntax highlighting, intelligent code completion, snippets, code refactoring, and embedded Git are among the features. Users can customize the theme, keyboard shortcuts, and preferences, as well as install extensions that offer new features.
Java, JavaScript, Go, Node.js, Python, and C++ are among the programming languages supported by VS Code. It's built on the Electron framework, which is used to create Node.js Web apps that leverage the Blink layout engine. It uses the same editing component as Azure DevOps (code-named "Monaco").

1.2 Database design tool

As a database design tool, we used MySQL Workbench, It is a visual tool that was made for developers and database administrators. This tool offers functionalities that allow the manipulation of database models, the execution of SQL queries and the configuration or the replication of databases.

1.3 Version Control System (VCS)

As VCS we used Git, It is a free and open source distributed version control system designed to keep track of code history. Git offers many features and is today's most popular VCS. Its main advantage is its branching feature which allows its users to organize the application development into a clear workflow by adapting a branching model. As a git online platform we used Github, it is the first online platform for hosting git repositories.

2 Development technologies

In This section, we will present the technologies used to implement this project, starting with the frontend technologies and ending with the backend technologies.

2.1 Presentation tier technologies

This part focuses on the technologies used for client development. The frontend must be extremely scalable. The web pages load time when already on the site should be minimal. Also, the display and processing of data has to be performed quickly and efficiently. In addition to that our project will be integrated in the already built Dnext platform so it's mandatory to respect the compatibility. In what follows, there will be a detailing of the choices made for the client development technologies.

2.1.1 React.JS

React is an open-source front-end JavaScript library for creating user interfaces and UI components. Facebook and a community of individual developers and businesses maintain it. React is just concerned with managing state and rendering it to the DOM. As a result, using React apps frequently necessitates the usage of extra libraries for routing and client-side functionality.
Components are the building blocks of React code. Using the React DOM library, components can be rendered to a specific element in the DOM [9].
The Document Object Model (DOM) is a cross-platform, language-independent interface that handles an XML or HTML content as a tree structure, with each node representing a component of the document as an object [10].

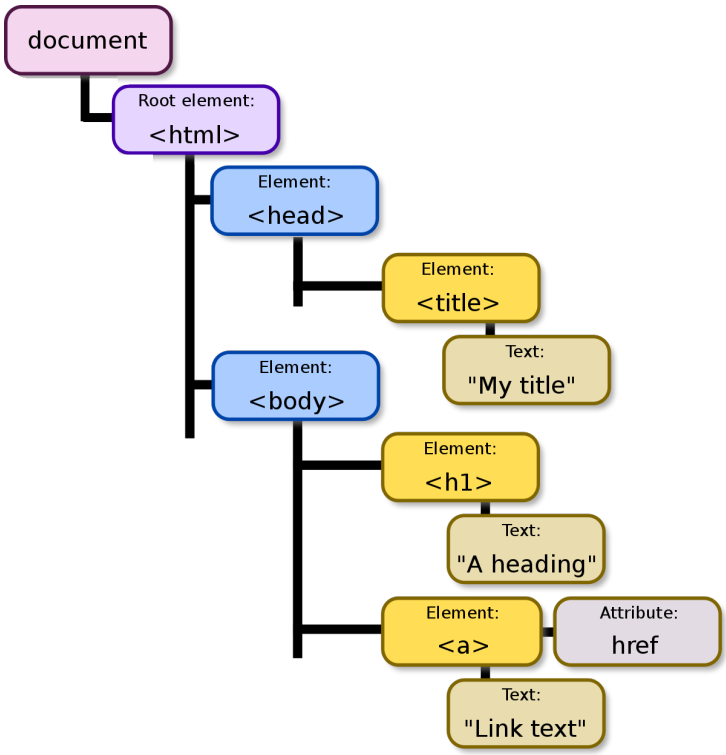


Figure 4.1: DOM hierarchy in an HTML document.

As for the routing, we used React Router. It is a standard library for routing. It allows users to navigate between views of different components in a React application, change the browser URL, and keep the UI in sync with it [11].

2.1.2 Ant Design

Ant Design is a React UI library with a variety of easy-to-use components for creating beautiful user interfaces. It was created by Alibaba, a Chinese company, and is utilized by a number of well-known companies, including Alibaba, Tencent, Baidu, and others. While Material-UI is the most popular React UI package, Ant Design is a close second and closing the gap swiftly [12].

2.1.3 HTML5, React Bootstrap and SCSS

- **HTML:** It is the default template creation language that React uses. The components have to define HTML tags to be called in other Templates.
- **React Bootstrap:** The Bootstrap JavaScript is replaced by React-Bootstrap. Each component was created from the ground up as a pure React component, with no un-

necessary dependencies such as jQuery. React-Bootstrap is one of the oldest React frameworks, and it has evolved alongside it, making it a fantastic UI basis [13].

- **SCSS:** It is an extension of CSS which is the default language for writing style sheets. SCSS adds the possibility of using variables and allows the declaration of functions.

2.2 Logic tier technologies

Although the Micro-services approach allows us to work with different technologies on the different micro-services, we chose to keep working with Node.js on the monitoring function, as it is the main technology used in the development of the Dnext API and all its services to ensure a better communication between the development team members and a faster development time.
Node.js is a cross-platform, open-source back-end JavaScript runtime environment that uses the V8 engine to execute JavaScript code outside of a web browser. It was first released in 2009 and continues to grow in popularity. Walmart, LinkedIn, Uber, NASA, and other large organizations have utilized this open-source, simple, and easy-to-customize environment [14].
The advantages of using Node.js:

- On high-load platforms, an I/O non-blocking architecture enables for fulfilling numerous simultaneous requests, allowing for easy scaling and rapid execution of client requests.
- The flexibility and responsiveness of the Node.js runtime environment are crucial attributes required by modern web apps, with an increasing number of online users seeking the most relevant information and services at their fingertips.
- Node.js is a sophisticated full-stack development solution. It lowers programmers' skill requirements and eliminates the need to learn new technologies.
- For backend development, the JavaScript programming language is particularly convenient and straightforward, inspiring software developers to abandon more sophisticated and cumbersome solutions. Node.js ensures that your software will not be obsolete in a few years and that other developers will be able to grasp your code easily.

2.3 Data tier technologies

- **MySQL Database:** MySQL is a relational database management system that is free and open-source (RDBMS). A relational database organizes data into one or more data tables in which data types can be associated to one another, allowing the data to be structured.
- **Dynamo Database:** DynamoDB is a NoSQL database,which is not tabular and doesn't hold data in the same way that relational tables do. NoSQL databases are classified according to their data model. Document, key-value, wide-column, and graph are the most common types. They have adaptable schemas and can handle big amounts of data and high user loads with ease.

3 Tests

The developed solution will be the subject of a continuous development process, since it encompasses the whole automated tasks. The upcoming changes may cause regressions. To ensure its proper functioning, a set of unit tests must be developed. The main object behind them is to test individual components and test the basic use cases. The Jest framework was used to implement the series of unit tests. Jest is a JavaScript testing framework maintained by Facebook. It was created with a focus on simplicity and support for massive web applications. It supports Babel, TypeScript, Node.js, React, Angular, Vue.js, and Svelte applications. For first-time users of a testing framework, Jest does not require much setting [15].
Figure 4.2 shows an example of a set of tests performed on the Links List Focus Point service.

```
PASS focusPoint/WebFocusPoint/LinksListChanges.test.js (5.359 s)
  all links with no filter
    √ should return all the links from AgroPolit page with no filter (98 ms)
  all links with filter
    √ should return all the links from AgroPolit page with Новини as filter (60 ms)
    √ should return link not found error  (46 ms)
  all links between two strings
    √ should return all the links from AgroPolit between 9 травня 2021, 10:10 and 29 квітня 2021, 12:00 (40 ms)
    √ should return string one not found error  (26 ms)
    √ should return an empty value error  (30 ms)

Test Suites: 1 passed, 1 total
Tests:       6 passed, 6 total
Snapshots:   0 total
Time:        6.08 s
```

Figure 4.2: Links List Focus Point Tests.

4 Application presentation

This section will present a complete workflow of the application. Starting with the main page, the listing and creation of the watchers, giving schedules and defining the notifications, then adding the focus points to finally showing the results of the execution. Figure 4.3 shows the dashboard of the Monitoring Module developed during this project.

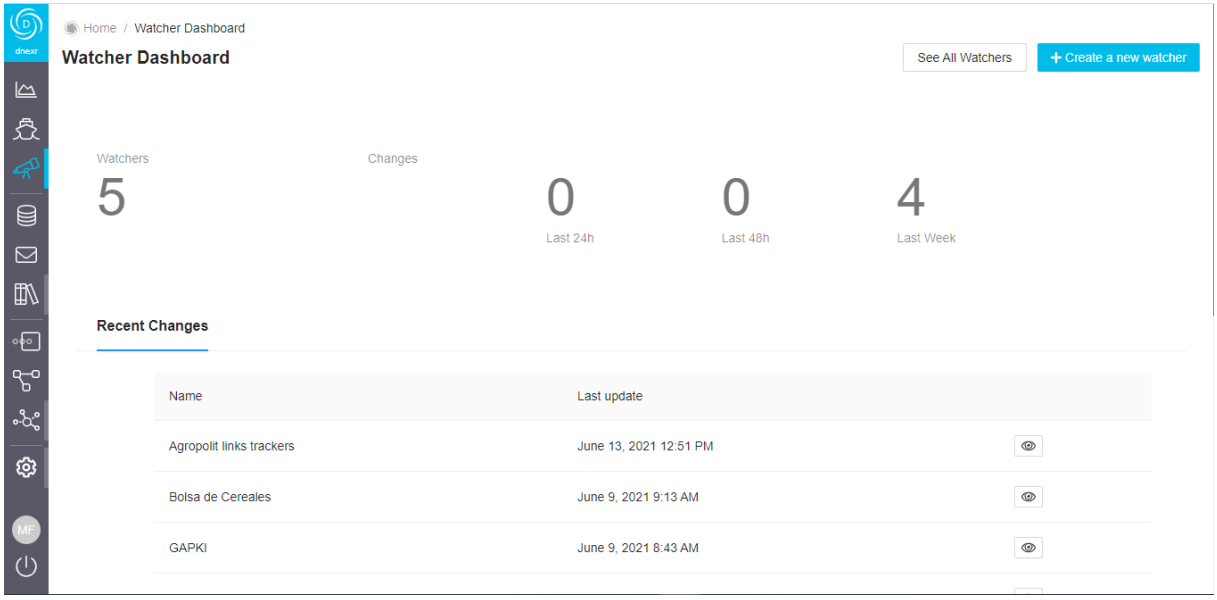


Figure 4.3: Main page of the Monitoring Module.

After pressing "See All Watchers" button the user is redirected to the watchers list page shown in Figure 4.4

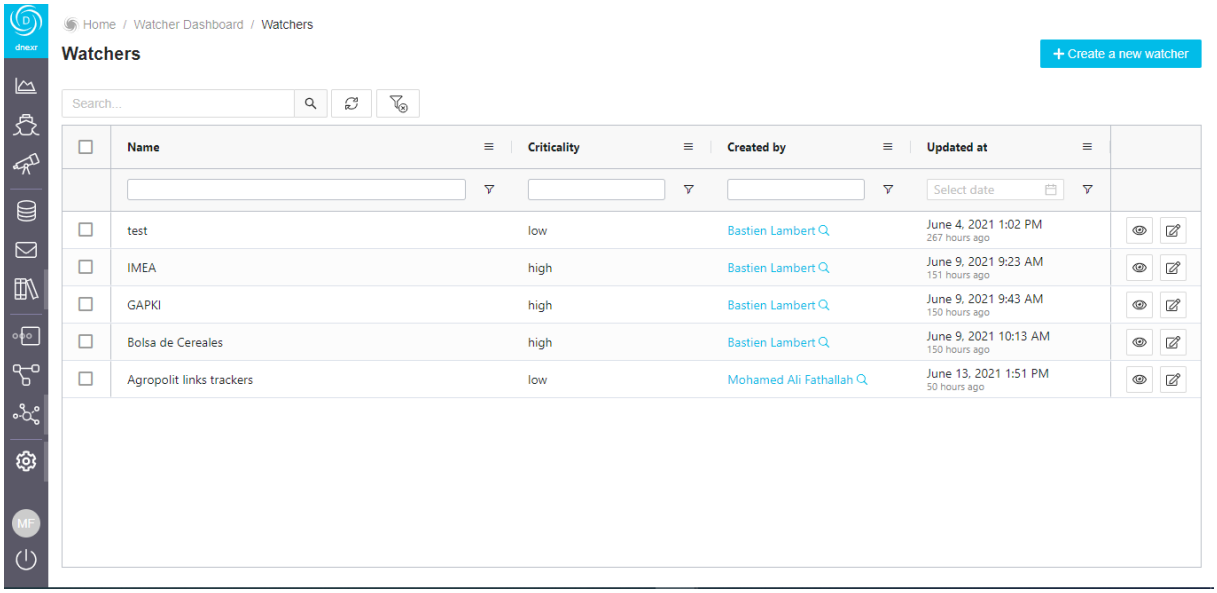


Figure 4.4: Watchers List Page.

To create a new watcher the user presses the "Create a new watcher" button, he will be redirected to the form shown in Figure 4.5 .

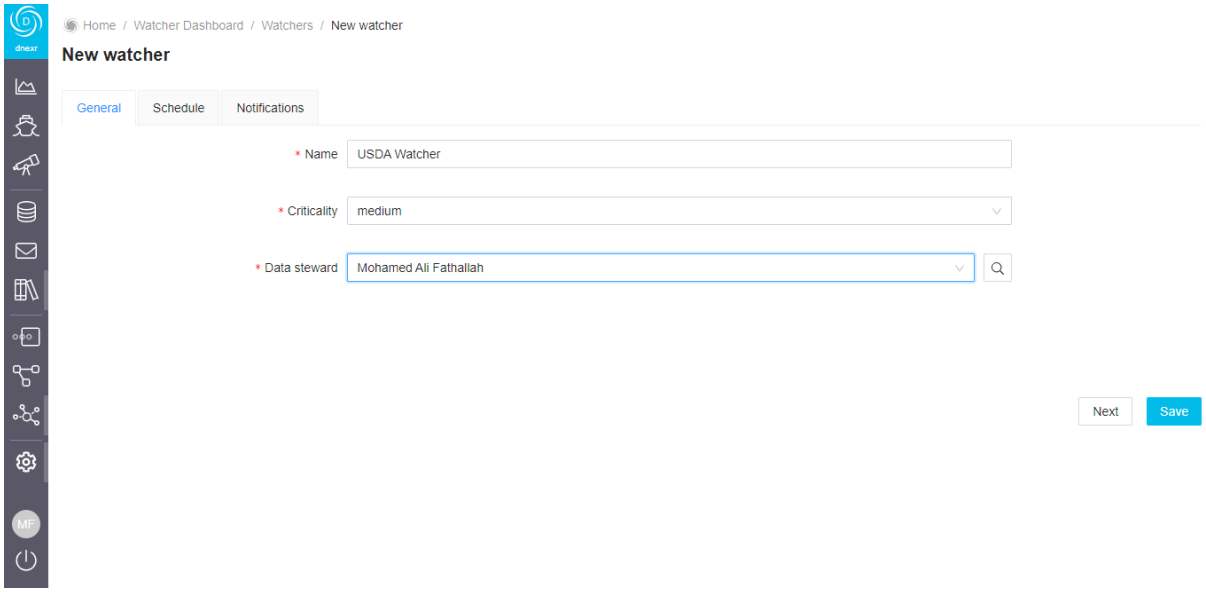


Figure 4.5: Watcher Creation Form .

After giving the watcher a name and a level of criticality and adding a data steward, he needs to add schedules on which the watcher will run, as shown in Figure 4.6

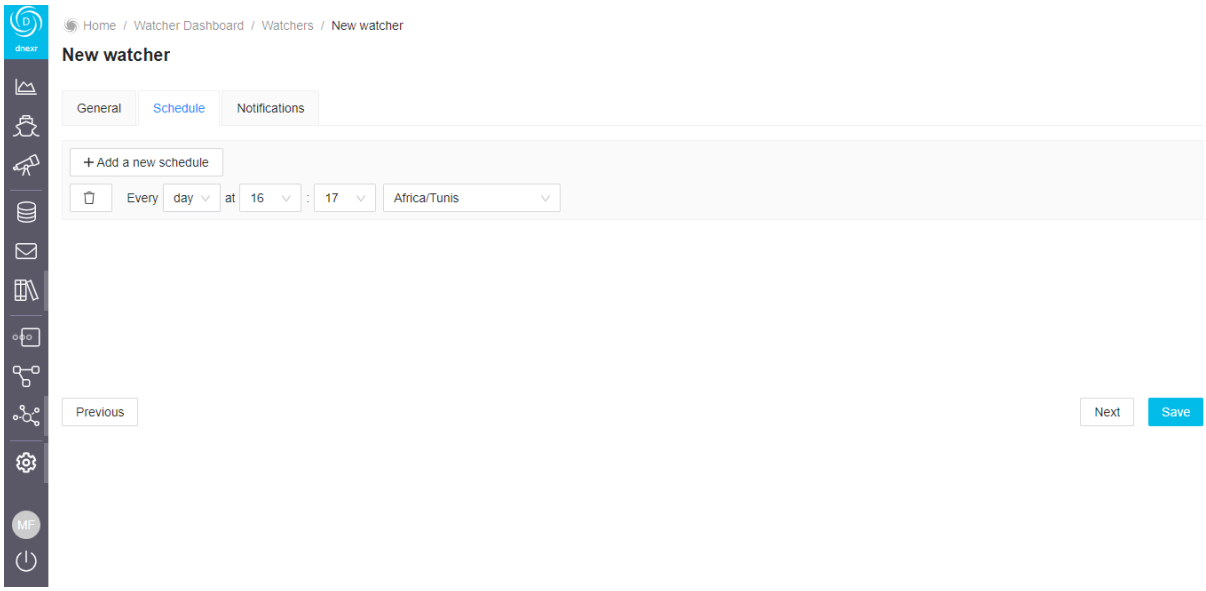


Figure 4.6: Adding Schedules Page.

After adding the schedules the user needs to define the notification options, as shown in Figure 4.7

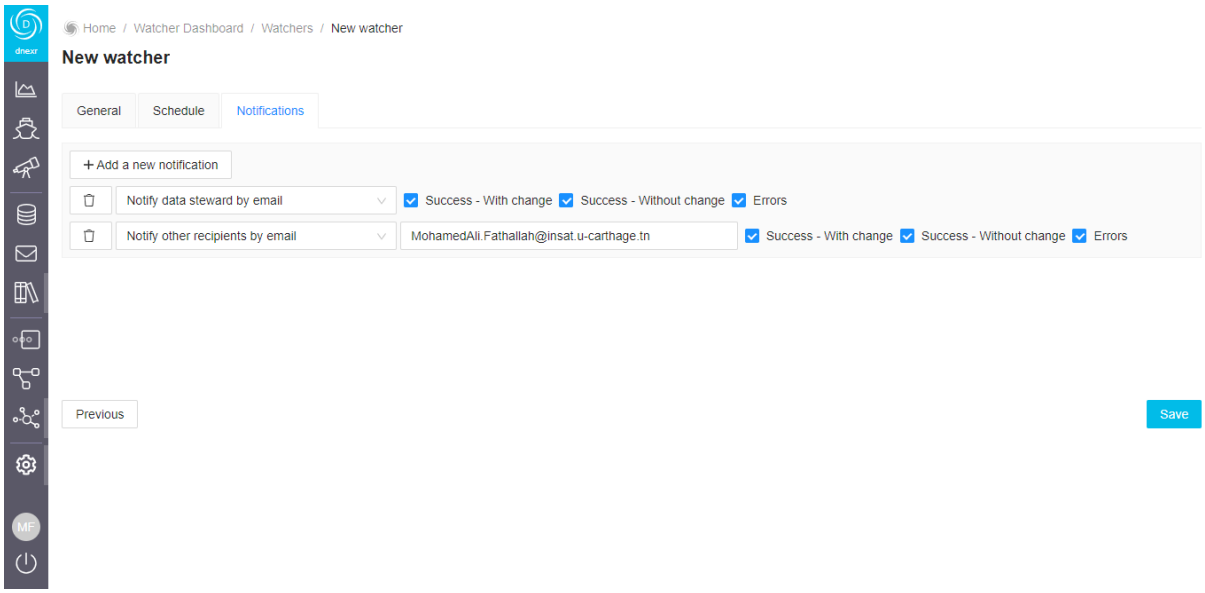


Figure 4.7: Adding Notifications Page.

Now that once the watcher is ready, the user presses the save button. Then, the watcher will be created and saved in the database and the user will be redirected to the watcher's details page shown in Figure A.1 in Appendix 2.

The user has to add, to the created watcher, some focus points to define which exact elements he wants to monitor So, he needs to press the 'create a new focus point' button and he will be redirected to the focus point creation form shown in Figure 4.8

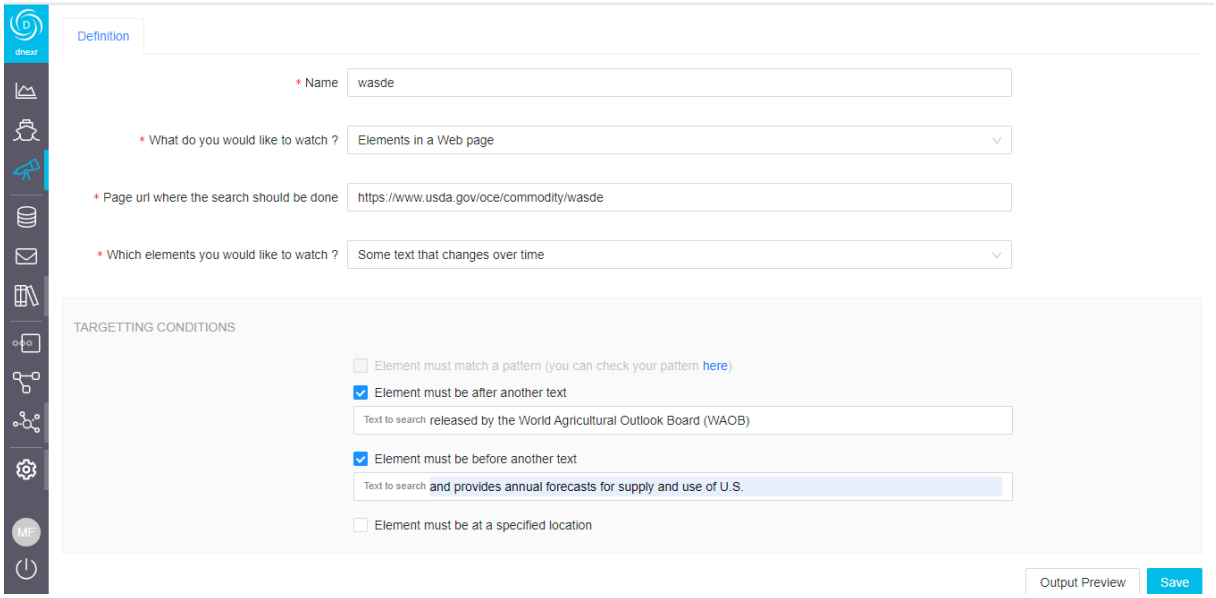


Figure 4.8: Focus Point Creation Form.

The user chooses the website and the monitoring type which is text update and uses a set of filters to select the exact element to monitor. Figure A.2 in Appendix 2. shows the element we want to monitor in the website.

Now that the focus point is ready the user can press 'save' to save it to the database and he will be redirected to the focus point details page shown in Figure A.4 in Appendix 2.

When the scheduled time arrives, the backend function will be executed automatically and will execute the watcher with its focus point. A notification will be sent, as shown in Figure 4.9 and the execution will be added in the execution history, as shown in Figure 4.10 with its value saved in the values history shown in Figure 4.11.

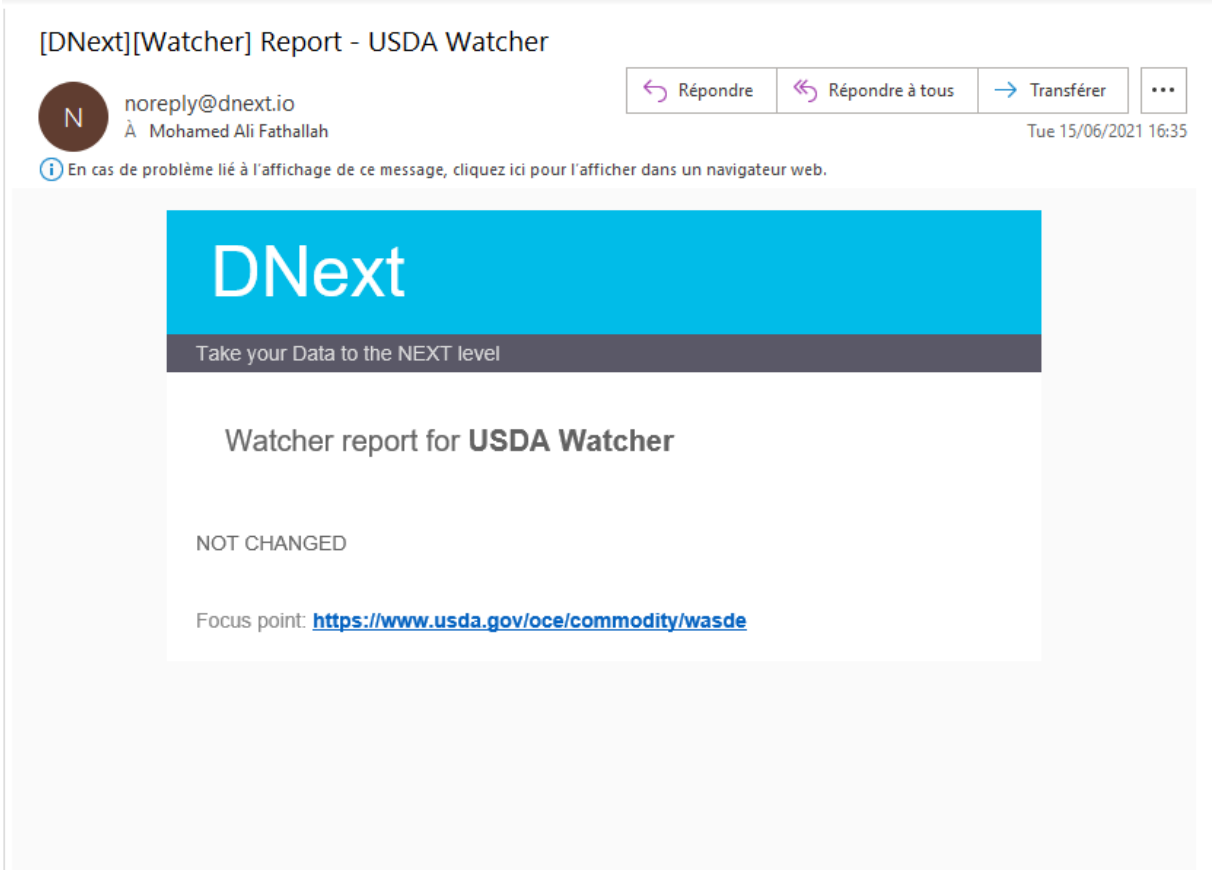


Figure 4.9: Email Notification.

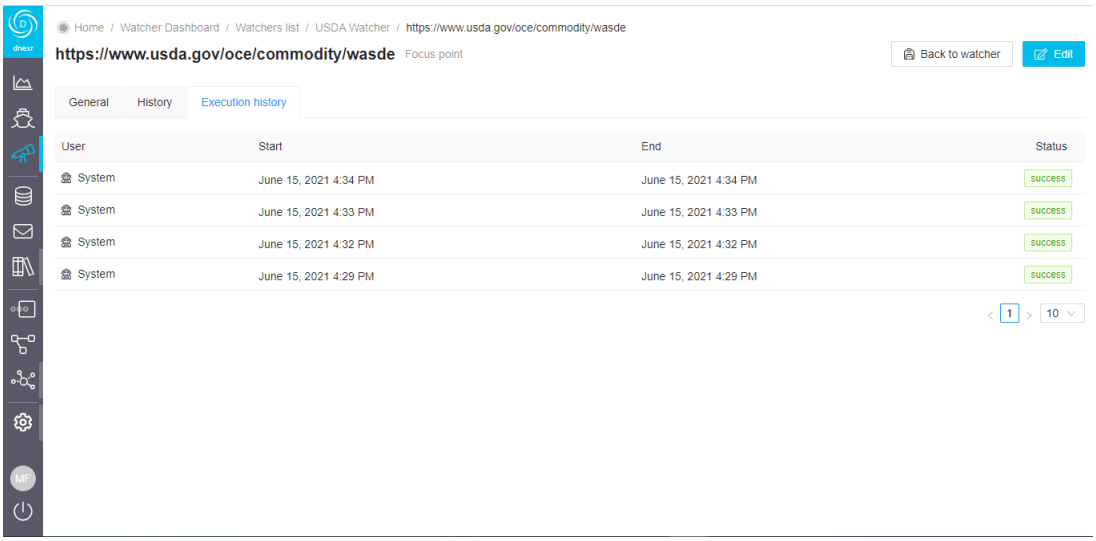


Figure 4.10: Focus Point Execution History.

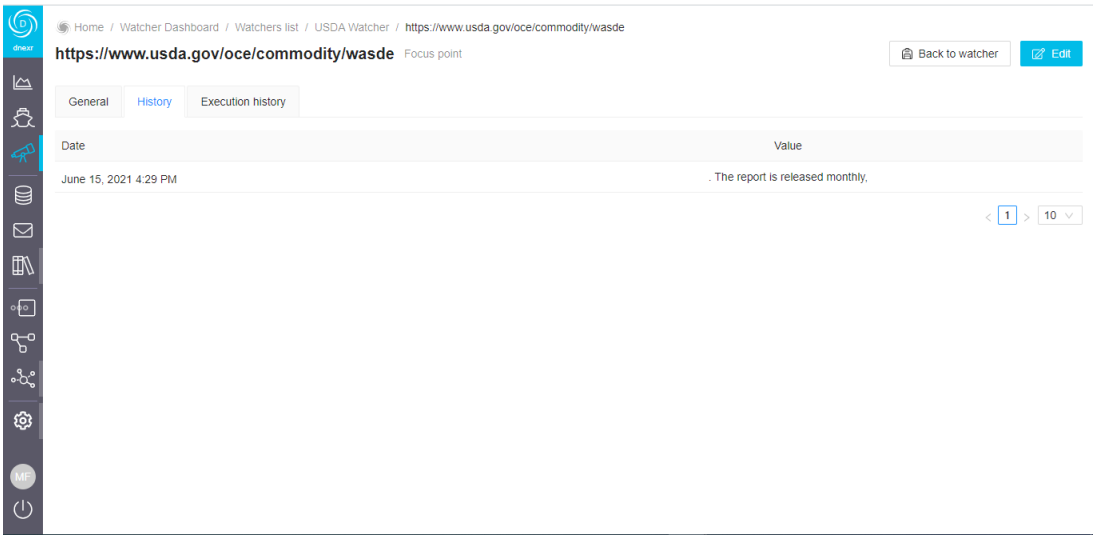


Figure 4.11: Focus Point Values History.

We note that the values history shows only the unique values.

Conclusion

This chapter covered the realization of the project. It started by presenting the software environment as well as the technologies used for the development of the project. Then, it moved to the tests module and finally it presented the different modules developed within the scope of the project. The next chapter will talk about the deployment.

5

Software deployment

Introduction

In this Chapter we will present the deployment environment chosen in the regard of the application architecture and then the deployment tool used.

1 Deployment environment

The Dnext platform used the AWS serverless framework as a deployment environment to adapt to the microservices architecture and that's because: [16]

- It scales with demand automatically.
- It significantly reduces server cost (70-90%).
- It eliminates server maintenance.
- It frees up developer resources to take on projects that directly drive business value (versus spending that time on maintenance).

1.1 AWS Serverless framework

Amazon Web Services (AWS) is a web-based platform that offers scalable and cost-effective cloud computing services.
AWS is a widely used cloud platform that helps businesses develop and thrive by providing on-demand services such as compute power, database storage, and content delivery [17].

Serverless is a cloud native design that allows you to delegate more of your operational tasks to AWS, resulting in increased agility and creativity. It enables you to create and run apps and services without having to consider servers. It takes care of infrastructure management chores like server or cluster provisioning, patching, and operating system upkeep. as well as capacity provisioning. You may create them for almost any form of application or backend service, you can run your application and scale it with high availability while everything you need to do that, is taken care of [18].

Figure 5.1 illustrates the general architecture of AWS Serverless.

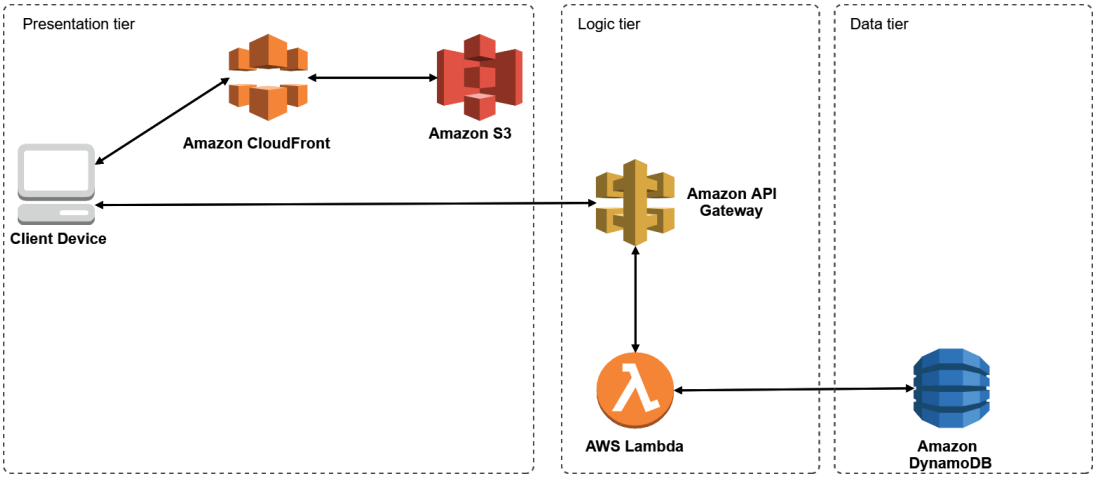


Figure 5.1: General AWS Serverless architecture

We can apply the 3-tiers approach on this architecture. We will further elaborate this approach in our application in what follows.

1.2 Project architecture

Here, we will present how we deployed the application using AWS Serverless framework with the 3-tiers approach. Figure 5.2 illustrates the general architecture of the application.

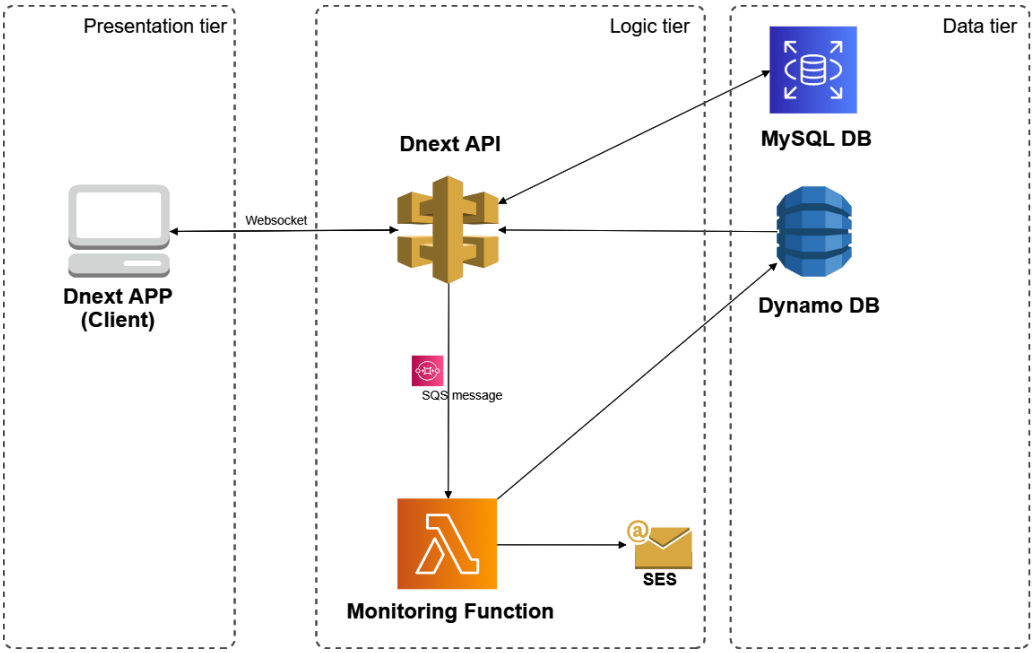


Figure 5.2: Project General Architecture

1.2.1 Presentation tier

The presentation tier consists of the Dnext platform web application, from the client side module. It contains the frontend of the project which is the user interface.
Figure 5.3 illustrates the detailed architecture of Dnext APP client.

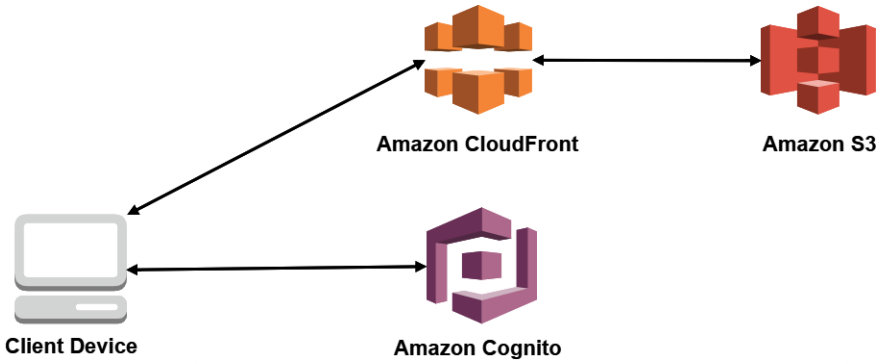


Figure 5.3: Dnext APP Client Architecture

The presentation tier can use many AWS serverless offerings, in our project it uses the following:

Content Delivery Network (Amazon CloudFront):
Amazon CloudFront is a developer-friendly content delivery network (CDN) service that securely distributes data, videos, applications, and APIs to consumers around the world with low latency and high transfer speeds.
CloudFront combines the most modern security features, such as field level encryption and HTTPS support,it is easily integrated with AWS Web Application Firewall, Amazon Route 53, and AWS Shield to protect against a variety of threats, including network and application layer DDoS attacks. These services coexist at global scaled edge networking sites connected via the AWS network.giving users a more safe, performant, and available experience.
Any AWS origin, such as Amazon EC2,Elastic Load Balancing, Amazon S3, or any custom HTTP origin, works flawlessly with CloudFront. CloudFront safe and programmable edge computing features allow the user to personalize his/her content delivery [19].

Web Host (Amazon S3):

It is used to host the platform's frontend static resources.

The Amazon Simple Storage Service is a cloud-based object storage service with industry-leading scalability, data availability, security, and performance. Amazon S3 is a simple web service interface that allows you to store and retrieve unlimited quantity of data from anywhere on the internet. This means it can be used by clients of various sizes and sectors to store and safeguard any amount of data for a variety of purposes. such as mobile applications ,enterprise applications , websites,backup and restore ,archive ,big data analytics, and IoT devices [20].

Identity Management (Amazon Cognito):

Amazon Cognito is an AWS product that manages user authentication and access for internet-connected applications. A user or visitor can sign in with a username and password through Amazon, or through a third party like Facebook, Google or Apple. The service saves and synchronizes end-user data, allowing an application developer to concentrate on writing code rather than constructing and maintaining back-end infrastructure. This can help to speed up the development of mobile apps.

Amazon Cognito organizes a user's profile attributes into user pools, which are used by the application to set up limited access to AWS resources. To arrange federated identity groups, an identity pool consolidates end-user information that is received by client access platforms, devices, and operating systems [21]. In conclusion, the two main components of Amazon Cognito are:

- **User pools:** User directories that provide sign-up and sign-in options for app users.
- **Identity pools:** Cognito elements grant users access to other AWS services.

1.2.2 Logic tier

This is where Amazon API Gateway and AWS Lambda can have the largest impact. The capabilities of these two services enable you to create a highly accessible, scalable, and secure serverless application. Your application might need thousands of servers in a classic or traditional model; but, by using Amazon API Gateway with AWS Lambda, you won't have to worry about server administration in any way. Thus Our Project's Logic Tier consists of 2 components:

- **Dnext API:**

  In the AWS serverless framework, we can use different types of API with the API Gateway, for example:

  - **REST API:** The API Gateway REST APIs use the request/response model, in which a client delivers a request to a service and the service responds synchronously. This approach is appropriate for a wide range of applications that rely on synchronous communication [22].
  - **Websocket API:** API Gateway WebSocket APIs are bidirectional. Clients can send messages to services, and services can send messages to clients on their own. Because services can push data to clients without requiring clients to make an explicit request, this bidirectional behavior allows for richer client/service interactions. Real-time apps frequently employ WebSocket APIs such as multiplayer games, chat applications, collaboration platforms, and financial trading platforms [23].

  Table 5.1 gives a comparison between the REST and Websocket APIs

Table 5.1: The difference between REST and Websocket APIs

The basis Of Comparison	**REST**	**WebSocket**
HTTP	HTTP is a common protocol in RESTful web services.	The use of HTTP occurs in the initial connection.
Communication	Uni-directional in nature.	Bi-directional in nature.
Nature	Resources based concept, rather than commands.	Socket-based concept.
Scenario	Lots of get requests.	Real-time chat application.
Dependency	Based on the HTTP protocol and uses HTTP methods to relay data.	Relies on IP address and port number.
Cost	The cost of communication is comparatively higher than WebSocket.	The cost of communication is lower.
Performance	Great for occasional communication.	Better with high loads.
State	REST is based on HTTP, which is a stateless protocol.	WebSocket is a stateful protocol.

Considering this comparison and in regard of Dnext's needs, our API is a websocket one.

Figure 5.4 shows a simplified architecture of Dnext API in the context of our project.

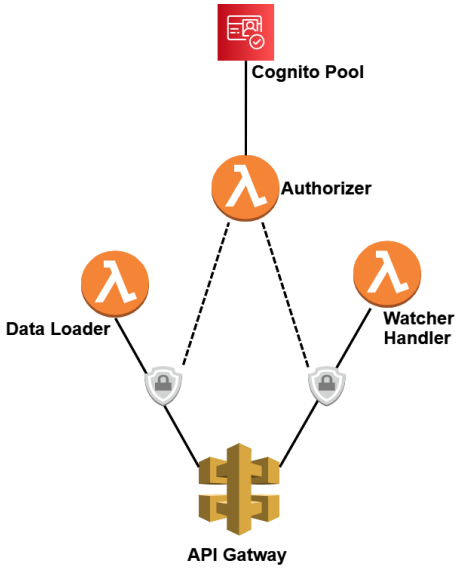


Figure 5.4: Dnext API Architecture

– The Authorizer function will manage the users authorizations and access rights to use the platform different services, it is connected to Cognito which contains the User and Identity Pools.

– The Data Loader will manage the data storing and loading as it is connected directly to the databases in the Data Tier

– The Watcher Handler is the main function to run the already created watchers, in regard of the schedules associated to every watcher, it will trigger the monitoring function which will do the backend work. this is done by sending a message to it containing the necessary arguments using Amazon Simple Queue Service (SQS) which is a fully managed message queuing service for decoupling and scaling microservices, distributed systems, and serverless applications.
SQS removes the complexity and costs of managing and operating message-oriented middleware, allowing developers to concentrate on work that is unique. You may send, store, and receive messages across software components using SQS at any volume without losing messages or necessitating the availability of other services [24].
Figure 5.5 shows the integration of a SQS queue between two lambda functions.



Figure 5.5: SQS queue integration

- **Monitoring Function:**

  We used Lambda functions to deploy the microservices.

  AWS Lambda is a computing service that lets you run arbitrary code functions in any supported language (Node.js, Python, Ruby, Java, Go, .NET) without having to provision, manage, or scale servers. Lambda functions run in a controlled, isolated container and are triggered by an event, which can be any of the programmatic triggers provided by AWS, known as an event source [25].

  The advantages of using Lambda:

  – No operating systems to choose, secure, patch, or manage.
  – No servers to right size, monitor, or scale.
  – Reduced risk to your cost from over-provisioning
  – Reduced risk to your performance from under-provisioning.

  This Monitoring function will be triggered by the Dnext API and it will execute a specific watcher, by running every focus point in regard of its monitoring type and filters, then it will store the results in a DynamoDB which is in the Data tier and send the necessary notifications using Amazon SES.

  SES (Amazon Simple Email Service) is a low-cost, flexible, and scalable email service that allows developers to send email from any application. Amazon SES can be swiftly configured to handle a variety of email use cases, including transactional, marketing, and bulk email communications. The customizable IP configuration and email authentication options available through Amazon SES assist to improve deliverability and safeguard sender reputation, while sending analytics track the impact of each email. You can send emails securely, globally, and at scale [26].

1.2.3 Data tier

The Data Tier contains all databases where we store the significant data for the project. The data storage options available in the data tier are unaffected by using AWS Lambda as your logic tier. By providing the proper database driver in the Lambda deployment package, Lambda functions can connect to any data storage choice. We mentioned the choices of data bases in chapter 4.

2 Deployment tool

As a deployment tool we used Terraform.

Terraform is an open-source infrastructure as code (IaC) software application built by HashiCorp. Users define and provide data center infrastructure using the HashiCorp Configuration Language (HCL) or JSON, which is a declarative configuration language.

Terraform is a tool that allows you to build, change, and version infrastructure in a secure and efficient manner. Terraform can manage both established and well-known service providers as well as unique in-house solutions. It uses configuration files to specify the components required to run a single application or an entire datacenter. Terraform creates an execution plan that explains how it will get to the target state and then executes it to build the infrastructure indicated. Terraform can determine what changed as the configuration changes and provide incremental execution plans that can be applied.

Terraform can manage both low-level and high-level infrastructure components, such as compute instances, storage, and networking, as well as DNS entries, SaaS features, and so on. The key features of Terraform are:

- Infrastructure as Code: A high-level configuration syntax is used to define infrastructure. This allows you to version your datacenter blueprint and treat it like any other piece of code. Infrastructure can also be shared and repurposed.
- Execution Plans: Terraform features a stage called "planning" that generates an execution plan. When you call apply, the execution plan shows what Terraform will perform. When Terraform manipulates infrastructure, this allows you to avoid any unpleasant surprises.
- Resource Graph: Terraform creates a graph of all your resources and parallelizes the

generation and change of any resources that aren't dependent on each other. Terraform constructs infrastructure as effectively as feasible as a result, and operators gain visibility into infrastructure dependencies.

- Change Automation: With minimal human input, complex changesets may be implemented to your infrastructure. With the execution plan and resource graph stated above, you'll know exactly what Terraform will modify and in what sequence, preventing numerous potential human errors [27].

Conclusion

This chapter covered the deployment of the project. It started by presenting the deployment environment as well as the technologies used to adapt to the architecture needed. And finally it presented the deployment tool used to build and manage this infrastructure.

Conclusion and perspectives

This graduation project, completed at Dnext aimed to improve of the Dnext Intelligence platform by developing a new module, namely Watcher. This new module allows the users of the platform to monitor websites, mainly those providing information and data on commodity markets news. Through this module, users are also notified when new records are posted or updated and can keep a history of those changes.

We conducted the project following the SCRUM management methodology. We started by analysing the existing Dnext Platform, then designing the envisioned solution by establishing the functional requirements. We set up the new module following a 3-tier architecture and we chose the tools and technologies adapted to our context.

As for our future work, we aim to add a new feature in this module which will allow to trigger automatically the Ingest data workflow when new records are published. The ingest data workflow is a module in Dnext platform that let the user import new data and transform it into a structured and significant datasets.

We, already, started working on this perspective, by writing python scripts that will scrape the data from different sources, web pages, APIs, PDFs, XL's,etc, and transform it into data frames. In that regard, we used many libraries, mainly requests, beautifulsoup4, zipfile, pandas and tabula.

A

Appendix 1: Use cases description

Table A.1: Create Watcher use case diagram description

Use Case	**Description**
Give name	The user has to choose a significant name to the watcher to give a general idea of which websites or elements it will be monitoring.
Define Criticality	The Criticality gives an idea how important is that watcher or how critical is the information it will be monitoring so it will be a way to distinguish watchers by their levels of importance.
Choose Data Steward	The Data Steward is the most concerned person by the information which the watcher will provide, generally he will receive all the notifications about it, Data Steward can be the user who created the watcher or any other allowed user who is part of the same organisation .
Add Schedules	The Schedule is the exact time when the watcher will be executed, the user chooses the day of the week, the exact hour and the timezone in that regard the watcher be executed periodically. the user can add multiple schedules.
Choose Notifications	The notifications are sent by email, the user choose who will receive them, it can be the data steward or other recipients, and when will they be sent,in case of success with new values, success without new values or error. the user can add multiples notifications.

Table A.2: Create Focus Point use case diagram description

Use Case	**Description**
Give name	The user has to choose a significant name to the focus point to give a general idea of which exact elements in the website it will be monitoring.
Choose monitoring type	The user has to choose the type of element that he wants to monitor, for example he can choose "text update" or "links list updates" as monitoring type.
Put URL	The user has to provide the URL of the website he wants to monitor .
Choose Filters	The user has to choose none or a combination of filters to narrow down the element he wants to monitor for example " the element exist after a certain string" or "the element contains a certain string".

A

Appendix 2: Platform presentation

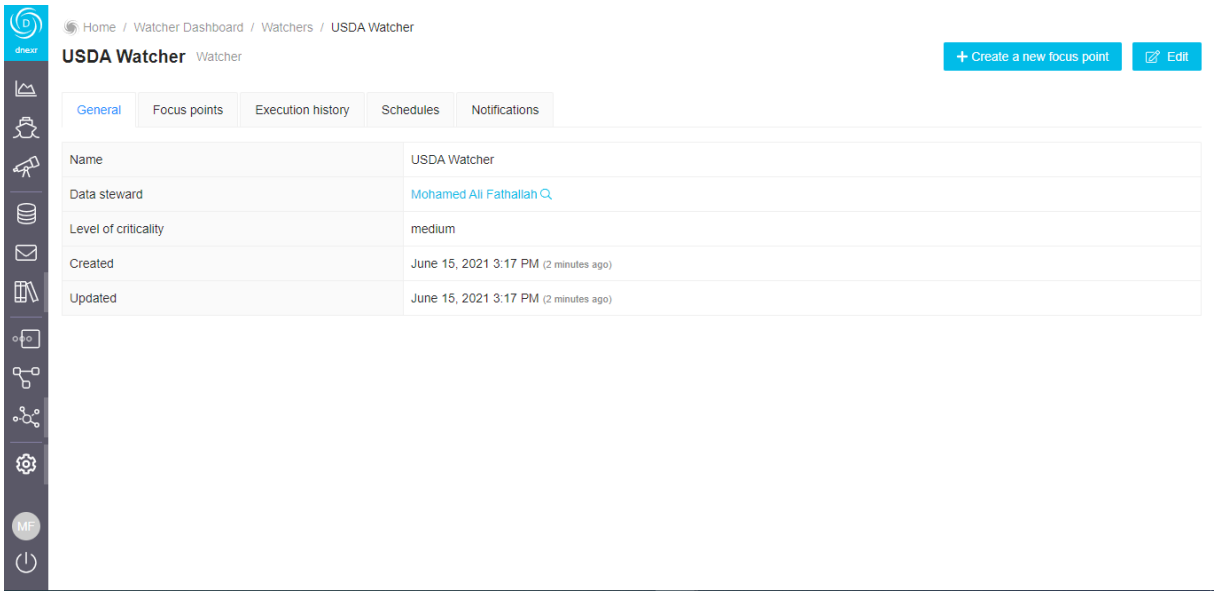


Figure A.1: Watcher Details Page.



Figure A.2: Element To Monitor.

Figure A.3 shows the result's preview of the focus point.

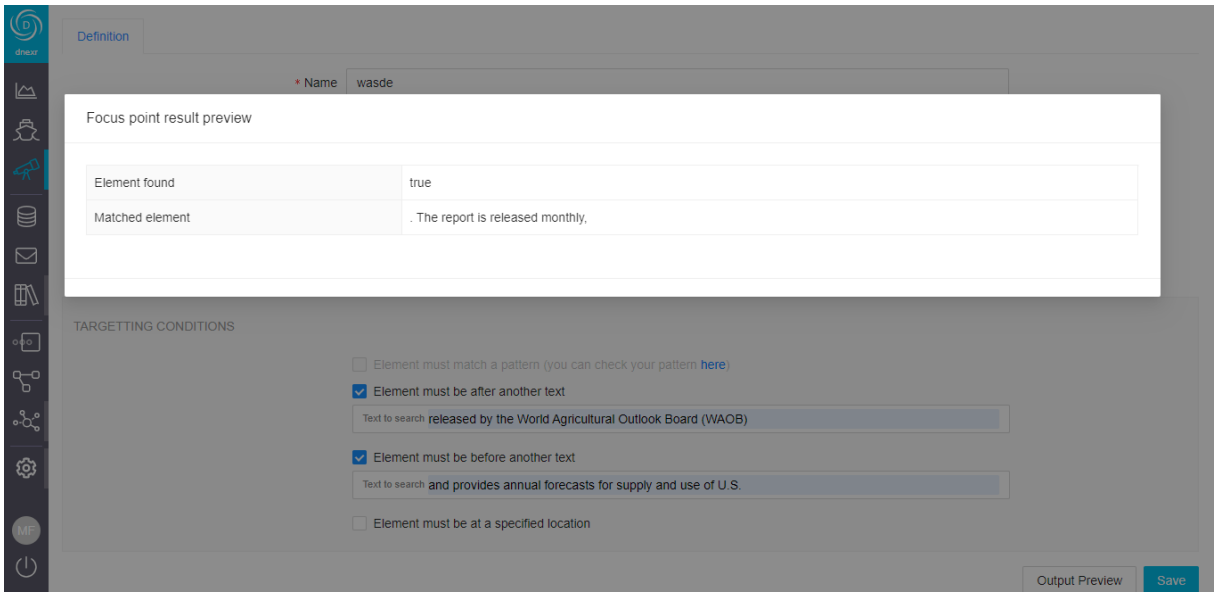


Figure A.3: Focus Point Result Preview.

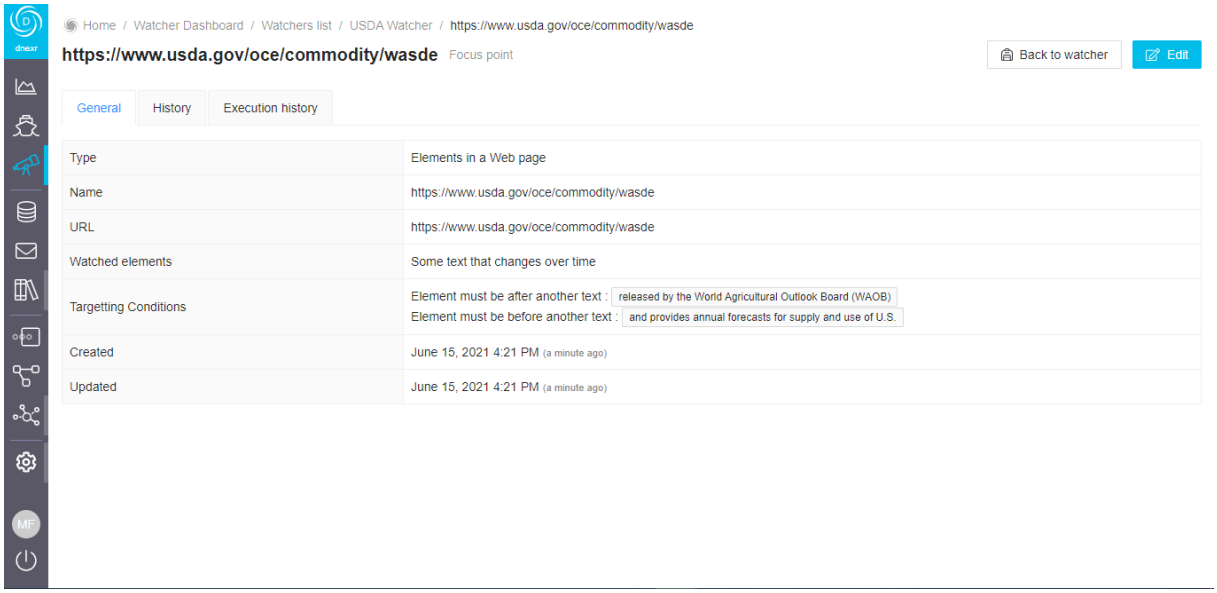


Figure A.4: Focus Point Details.

Bibliography

[1] Dnext website. https://www.dnext.io/. visited on 18/04/2021.

[2] Agile vs traditional methodologies. https://www.kpipartners.com/blog/traditional-vs-agile-software-development-methodologies#:~:text=The%20main%20difference%20between%20traditional,in%20Agile%2C%20it%20is%20iterative. visited on 22/04/2021.

[3] Agile methodologies. http://blog.daverooney.ca/2008/11/waterfall-vs-rad-vs-rup-vs-agile-scrum.html. visited on 22/04/2021.

[4] Agile scrum. https://www.atlassian.com/agile/scrum. visited on 22/04/2021.

[5] Microservice architecture. https://microservices.io. visited on 18/05/2021.

[6] Aws serverless multi-tier architectures. https://docs.aws.amazon.com/whitepapers/latest/serverless-multi-tier-architectures-api-gateway-lambda/introduction.html. visited on 29/05/2021.

[7] Content delivery network. https://www.cloudflare.com/learning/cdn/what-is-a-cdn/. visited on 18/05/2021.

[8] Identity management. https://searchsecurity.techtarget.com/definition/identity-management-ID-management#:~:text=Identity%20management%20(ID%20management)%20is,to%20applications%2C%20systems%20or%20networks. visited on 18/05/2021.

[9] React.js. https://reactjs.org/. visited on 23/05/2021.

[10] Document object model. shorturl.at/bnA16. visited on 23/05/2021.

[11] React router. https://reactrouter.com/. visited on 23/05/2021.

[12] Ant design. https://ant.design/. visited on 24/05/2021.

[13] React bootstrap. https://react-bootstrap.github.io/. visited on 24/05/2021.

[14] Node.js. https://nodejs.org/en/. visited on 26/05/2021.

[15] Jestjs. https://jestjs.io/. visited on 26/05/2021.

[16] When to use aws serverless. https://www.serverless.com/blog/when-why-not-use-serverless. visited on 28/05/2021.

[17] Amazon web services. https://aws.amazon.com/what-is-aws/. visited on 28/05/2021.

[18] Aws serverless. https://aws.amazon.com/lambda/serverless-architectures-learn-more/. visited on 28/05/2021.

[19] Aws cloudfront. https://aws.amazon.com/cloudfront/. visited on 29/05/2021.

[20] Aws s3. https://aws.amazon.com/s3/. visited on 29/04/2021.

[21] Amazon cognito. https://searchaws.techtarget.com/definition/Amazon-Cognito. visited on 29/05/2021.

[22] Rest api. https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-rest-api.html. visited on 01/06/2021.

[23] Websocket api. https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-websocket-api.html. visited on 21/06/2021.

[24] Amazon sqs. https://aws.amazon.com/sqs/. visited on 01/06/2021.

[25] Amazon lambda. https://aws.amazon.com/lambda/. visited on 02/06/2021.

[26] Amazon ses. https://aws.amazon.com/ses/. visited on 02/06/2021.

[27] Terraform. https://www.terraform.io/intro/index.html. visited on 03/06/2021.

العنوان : تطوير وحدة مراقبة وتنبيه آلية للإنترنت لمنصة Dnext

ملخّص : يتعلق هذا العمل بتحسين منصة Dnext من خلال تطوير وحدة نمطية جديدة، وهي مراقب. تهدف هذه الوحدة الجديدة إلى السماح لمستخدمي المنصة مراقبة المواقع الإلكترونية، وخاصة تلك التي توفر المعلومات والبيانات عن أسواق السلع الأساسية خبر. من خلال هذه الوحدة النمطية، يتم إعلام المستخدمين أيضا عند نشر سجلات جديدة أو تحديث ويمكن الاحتفاظ بتاريخ هذه التغييرات يتم تطوير المشروع على مدار سلسلة من المراحل بدءا من خلال تحقيق ما يلزم متطلبات التطبيق وتصميم الحل المتوخى. هذه الدراسة ستكون تليها إنشاء هيكل فني مفصل للحل وأخيرا تنفيذ التطبيق ونشره .

المفاتيح : أوس ,بدون خادم , ريكت ,ند ,مكدس كامل ,ترفورم , أب ي, ماي اسكيال, دينامو

Titre : Développement d'un module automatisé de surveillance et d'alerte Internet pour la plateforme Dnext

Résumé: Ce travail concerne l'amélioration de la plateforme Dnext Intelligence par le développement d'un nouveau module Watcher. Ce nouveau module vise à permettre aux utilisateurs de la plateforme de surveiller les sites web, principalement ceux qui fournissent des informations et des données sur l'actualité des marchés des matières premières. Grâce à ce module, les utilisateurs sont également avertis lorsque de nouveaux enregistrements sont publiés ou mis à jour et peuvent conserver un historique de ces changements. Le projet est développé à travers une série d'étapes commençant par la déduction des spécifications nécessaires de l'application et la conception de la solution envisagée. Cette étude sera suivie par l'établissement d'une structure technique détaillée de la solution et enfin la mise en oeuvre et le déploiement de l'application.

Mots-clés: AWS, Serverless, Node.js, React, Full stack, Terraform, API, MySQL, Dynamo

Title: Development of an automated Internet monitoring and alert module for Dnext platform

Abstract: This work concerns the improvement of the Dnext Intelligence platform by developing a new module, namely Watcher. This new module aims to allow the users of the platform to monitor websites, mainly those providing information and data on commodity markets news. Through this module, users are also notified when new records are posted or updated and can keep a history of those changes. The project is developed throughout a series of stages starting by deducing the necessary requirements of the application and designing the envisioned solution. This study will be followed by an establishment of a detailed technical structure of the solution and finally the implementation and deployment of the application.

Key Word: AWS, Serverless, Node.js, React, Full stack, Terraform, API, MySQL, Dynamo

Intitule et adresse complète de l'entreprise :

Entreprise : Dnext Intelligence SA
Adresse : Avenue Louis-Casaï 81 CH - 1216 Cointrin
Tél. : +41 22 732 53 72
Fax :
Email : info@dnext.io